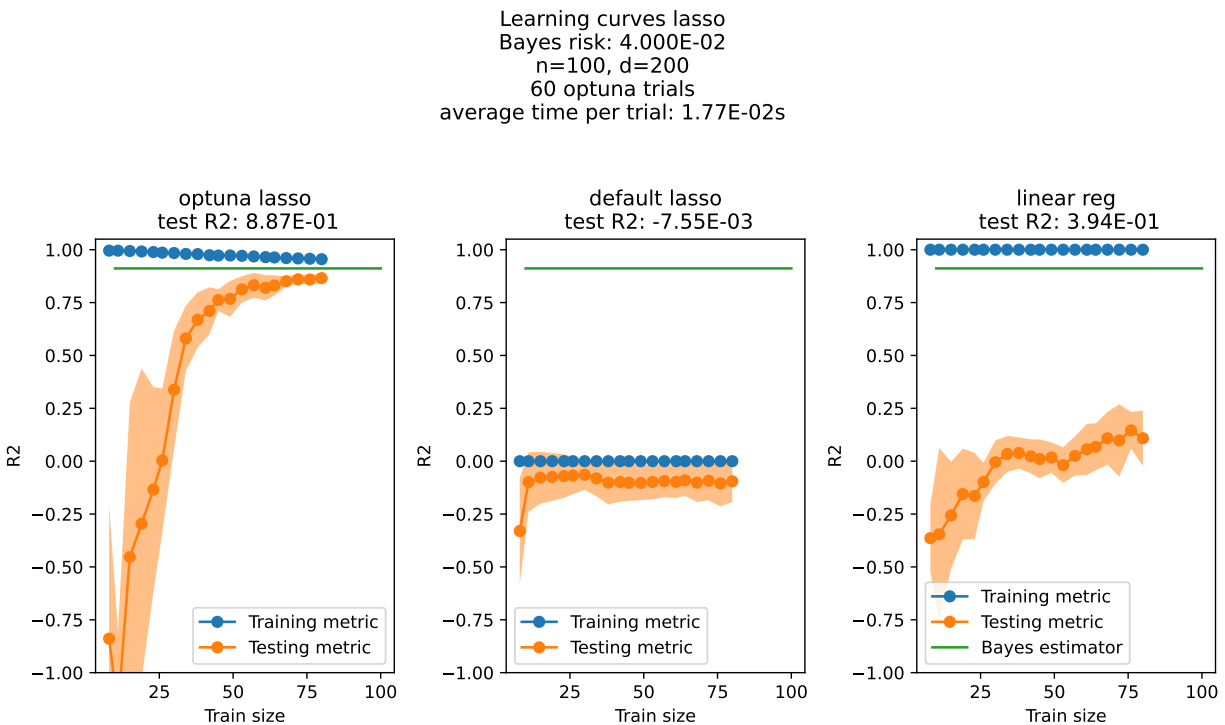
Fondamentaux théoriques du machine learning



Notion of risk decomposition

Context : usual supervised learning.

- empirical risk of estimator f : $R_n(f)$ (random variable)
- risk (real risk, generalization error) of estimator f : $R(f)$ (real number).
- dataset D_n.

We would like to interpret the risk of any estimator.

Convergence of empirical risk

We fix $f \in H$ (hypothesis space). We assume that the samples (x_i, y_i) are i.i.d, with the distribution of (X, Y), noted ρ. Then, according to the law of large numbers, under some assumptions (for instance, if the empirical risks are bounded), we have proven that in probability :

$$\lim_{n \to +\infty} R_n(f) = R(f) \tag{1}$$

The empirical risk of a fixed f converges to its real risk.

Notion of risk decomposition

We would like to compare the real risk $R(g)$, for some estimator g, to $R^* = R(f^*)$, f^* being the Bayes estimator.

First decomposition

- f^* : Bayes predictor
- F : Hypothesis space
- g : some predictor ($\in F$).

$$R(g) - R^* = \Big(R(g) - \inf_{f \in F} R(f)\Big) + \Big(\inf_{f \in F} R(f) - R^*\Big) \qquad (2)$$

First decomposition

- f^* : Bayes predictor
- F : Hypothesis space
- g : some predictor
- f_n : empirical risk minimizer

Most of the time, we will be interested in such a decomposition for the learned estimator, e.g. the empirical risk minimizer f_n. Now, $R(f_n)$ is a **random variable** !

$$E[R(f_n)] - R^* = \Big(E[R(f_n)] - \inf_{f \in F} R(f)\Big) + \Big(\inf_{f \in F} R(f) - R^*\Big) \quad (3)$$

Underfitting and overfitting

Approximation error (bias) : depends on f^* and F, not on f_n, D_n.

$$\inf_{f \in F} R(f) - R^* \geq 0$$

Estimation error (variance, fluctuation error, stochastic error) : depends on D_n, F, f_n.

$$E(R(f_n)) - \inf_{f \in F} R(f) \geq 0$$

- too small F (compared to f^*) : underfitting (large bias, small variance)
- too large F (compared to n) : overffitting (small bias, large variance)

Deterministic bound on the estimation error

We consider the best estimator in the hypothesis space F.

$$f_a = \arg\min_{h \in F} R(h)$$

Exercice 1 : Show that

$$R(f_n) - R(f_a) \leq 2 \sup_{h \in F} |R(h) - R_n(h)| \tag{4}$$

Deterministic bound on the estimation error

$$R(f_n) - R(f_a) \leq 2 \sup_{h \in F} |R(h) - R_n(h)| \tag{5}$$

Later in the course, based on **concentration inequalities** we will further build on this result and prove a probabilistic bound of the form

$$R(f_n) - R(f_a) \leq \frac{C}{\sqrt{n}} \tag{6}$$

(remember that by definition $0 \leq R(f_n) - R(f_a)$)

Approximate solution

- In machine learning, it is often not necessary to find the actual minimizer of the empirical risk, as there is an estimation error of $\mathcal{O}(\frac{1}{\sqrt{n}})$. [Bottou and Bousquet, 2009,]
- We can use an approximate solution $\hat{f}_n$, such that

$$R_n(\hat{f}_n) \leq R_n(f_n) + \rho \tag{7}$$

  with ρ a predefined tolerance.
- This important because in large-scale ML, the **computation time** needs to be optimized.

Approximate solution

This gives a new risk decomposition :

$$\begin{aligned} E[R(\hat{f}_n)] - R^* = & \left(E[R(\hat{f}_n)] - E[R(f_n)] \right) \\ & + \left(E[R(f_n)] - \inf_{f \in F} R(f) \right) \\ & + \left(\inf_{f \in F} R(f) - R^* \right) \end{aligned} \tag{8}$$

$$
\begin{aligned}
E[R(\hat{f}_n)] - R^* = & \Big(E[R(\hat{f}_n)] - E[R(f_n)] \Big) \\
& + \Big(E[R(f_n)] - \inf_{f \in F} R(f) \Big) \\
& + \Big(\inf_{f \in F} R(f) - R^* \Big)
\end{aligned} \tag{9}
$$

$E[R(\hat{f}_n)] - E[R(\tilde{f}_n)]$ is the **optimization error.**
To conclude, we have :

- an approximation error
- an estimation error
- an optimization error

Deterministic bound on the estimation error

We will now focus on the estimation error

$$E\Big[R(f_n)\Big] - \inf_{f \in F} R(f) \geq 0$$

f_n is the empirical risk minimizer
We consider the best estimator in hypothesis space

$$f_a = \arg\min_{h \in F} R(h)$$

We have seen that

$$R(f_n) - R(f_a) \leq 2 \sup_{h \in F} |R(h) - R_n(h)| \tag{10}$$

Deterministic bound on the estimation error

$$f_a = \arg\min_{h \in F} R(h)$$

$$f_n = \arg\min_{h \in F} R_n(h) \tag{11}$$

$$\begin{aligned} R(f_n) - R(f_a) = &\big(R(f_n) - R_n(f_n)\big) \\ &+ \big(R_n(f_n) - R_n(f_a)\big) \\ &+ \big(R_n(f_a) - R(f_a)\big) \end{aligned} \tag{12}$$

But by definition f_n minimizes R_n, so $\big(R_n(f_n) - R_n(f_a)\big) \leq 0$.
Finally :

$$R(f_n) - R(f_a) \leq 2 \sup_{h \in F} |R(h) - R_n(h)| \tag{13}$$

Bound on the estimation error

If we are able to bound $\sup_{h \in F} |R(h) - R_n(h)|$, then we have a bound on the estimation error.

Deterministic bound on the estimation error

Theorem

Weak law of large numbers
Let $(X_i)_{i\in\mathbb{N}}$ be a sequence of i.i.d. variables that have a moment of order 2. We note m their expected value. Then

$$\forall \epsilon > 0, \lim_{n\to+\infty} P\left(\left|\frac{1}{n}\sum_{i=1}^{n} X_i - m\right| \geq \epsilon\right) = 0 \tag{14}$$

*We say that we have **convergence in probability**.*

However, this is only an asymptotical result : it is a limit for $n \to +\infty$.

Bound on the estimation error

If we are able to bound $\sup_{h \in F} |R(h) - R_n(h)|$, then we have a bound on the estimation error.
To do this, we will use some other mathematical results :

- Boole's inequality
- Hoeffding's inequality (non-asymptotical probabilistic bound)

Boole's inequality

Proposition

Let $A_1, A_2, \ldots,$ be acountable set of events of a probability space $\{\Omega, \mathcal{F}, P)$.
Then,

$$P\Big(\cup_{i \geq 1} A_i \Big) \leq \sum_{i \geq 1} P(A_i) \tag{15}$$

This set might be infinite.

Boole's inequality

Proposition

Let $A_1, A_2, \ldots,$ be acountable set of events of a probability space $\{\Omega, \mathcal{F}, P)$.
Then,

$$P\Big(\cup_{i\geq 1} A_i\Big) \leq \sum_{i\geq 1} P(A_i) \tag{16}$$

This set might be infinite.
Exercice 2 : Proove the proposition for tomorrow's practical session, in the case of a finite set of A_i, and optionnally for an infinite set.

Hoeffding's inequality

Theorem

Hoeffding's inequality
Let $(X_i)_{1 \leq i \leq n}$ *be n i.i.d real random variables such that* $\forall i \in [1, n]$, $X_i \in [a, b]$ *and* $E(X_i) = \mu \in \mathbb{R}$. *Let* $\bar{\mu} = \frac{1}{n} \sum_{i=1}^{n} X_i$.
Then $\forall \epsilon > 0$,

$$P\Big(|\bar{\mu} - \mu| \geq \epsilon\Big) \leq 2 \exp\Big(-\frac{2n\epsilon^2}{(b-a)^2}\Big)$$

We admit this theorem.

Example

- If $d >> n$ and we want to learn a linear model $x \mapsto \langle \theta, x \rangle$, we have seen that this raises statistical issues (high variance, overfitting).
- However, if we know in advance that θ only has $s < d$ non-zero coordinates (sparse θ), we can reformulate to an easier problem.
- But most of the time this is not the case, s is not known, so we need to test several subsets of non-zero coordinates.

Example

We could write the following regularized optimization problem

$$\hat{\theta} = \underset{\theta \in \mathbb{R}^d}{\arg\min} \left(||y - X\theta|| + \lambda ||\theta||_0 \right) \tag{17}$$

- $y \in \mathbb{R}^n$ (labels)
- $X \in \mathbb{R}^{n,d}$ (design matrix)
- $||\theta||_0$: number of non-zero components of θ

Example

We could write the following regularized optimization problem

$$\hat{\theta} = \underset{\theta \in \mathbb{R}^d}{\arg\min} \left(||y - X\theta|| + \lambda ||\theta||_0 \right) \tag{18}$$

- $y \in \mathbb{R}^n$ (labels)
- $X \in \mathbb{R}^{n,d}$ (design matrix)
- $||\theta||_0$: number of non-zero components of θ

However,

- optimization issue (not convex)
- computationally prohibitive to test all subsets of $[1, d]$.
  Exercice 3 : How many subsets does $[1, d]$ contain ?

Lasso

Le Lasso replaces $||\theta||_0$ by $||\theta||_1$.

$$||\theta_1|| = \sum_{i=1}^{d} |\theta_i| \tag{19}$$

Lasso estimator :

$$\tilde{\theta}_\lambda \in \underset{\theta \in \mathbb{R}^d}{\arg\min} \{||y - X\theta||^2 + \lambda ||\theta||_1\} \tag{20}$$

For technically involved reasons, the optimization with the lasso leads to sparser solutions in some situations.

Lasso

Lasso estimator :

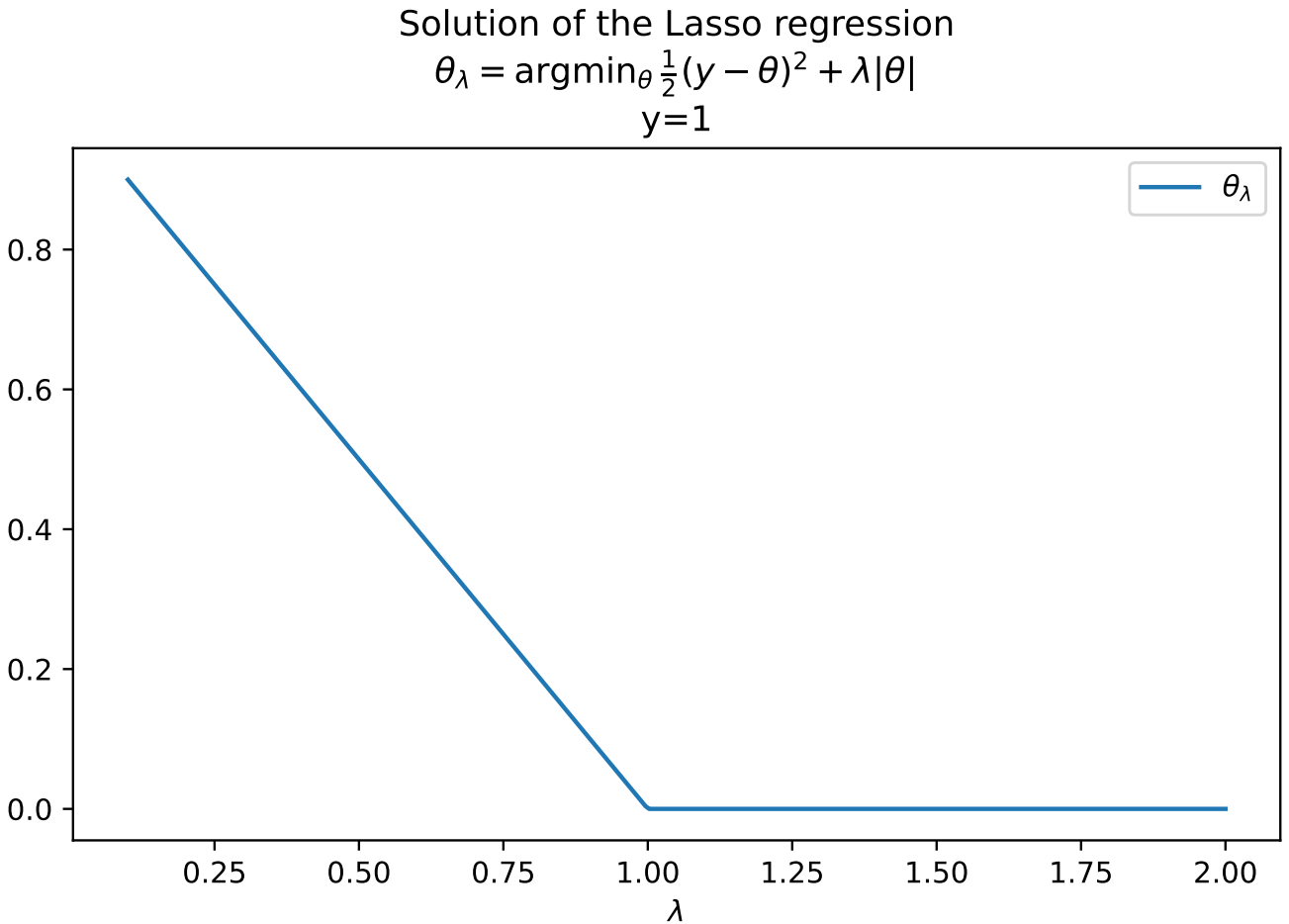
$$\tilde{\theta}_\lambda \in \underset{\theta \in \mathbb{R}^d}{\arg\min}\{||y - X\theta||^2 + \lambda||\theta||_1\} \tag{21}$$

For technically involved reasons, the optimization with the lasso leads to sparser solutions. Frequently used optimization algorithm :

- coordinate descent (algorithm used in scikit)
- Fista
- LARS

`https://en.wikipedia.org/wiki/Coordinate_descent`

Example in 1D : TP 12



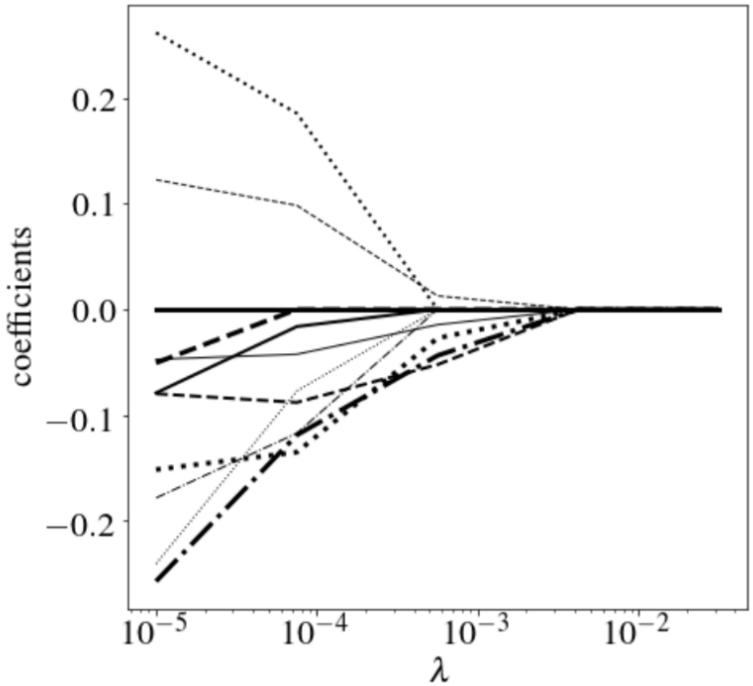


Figure – Regularization path with a Lasso optimization of a problem with $d = 12$. Each line represents the evolution of a θ_i when λ increases. Image from [Azencott, 2022].

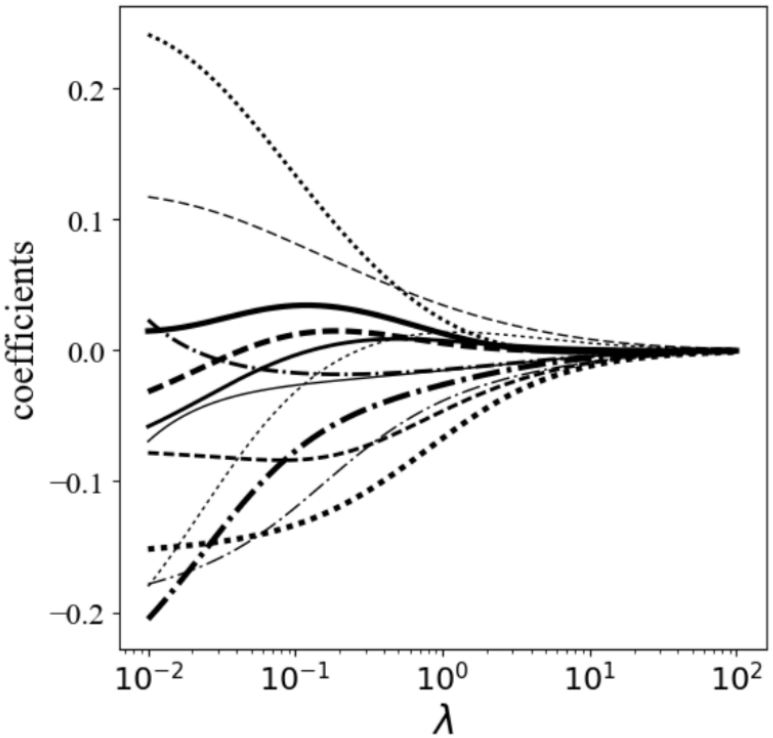


Figure – Regularization path with a Ridge optimization of a problem with $d = 12$. Each line represents the evolution of a θ_i when λ increases. Image from [Azencott, 2022].

Elastic net

Combination of $L1$ and $L2$ regularization.
Elastic-net estimator :

$$\tilde{\theta}_\lambda \in \underset{\theta \in \mathbb{R}^d}{\arg\min}\{||y - X\theta||^2 + \lambda_1||\theta||_1 + \lambda_2||\theta||_2^2\} \qquad (22)$$

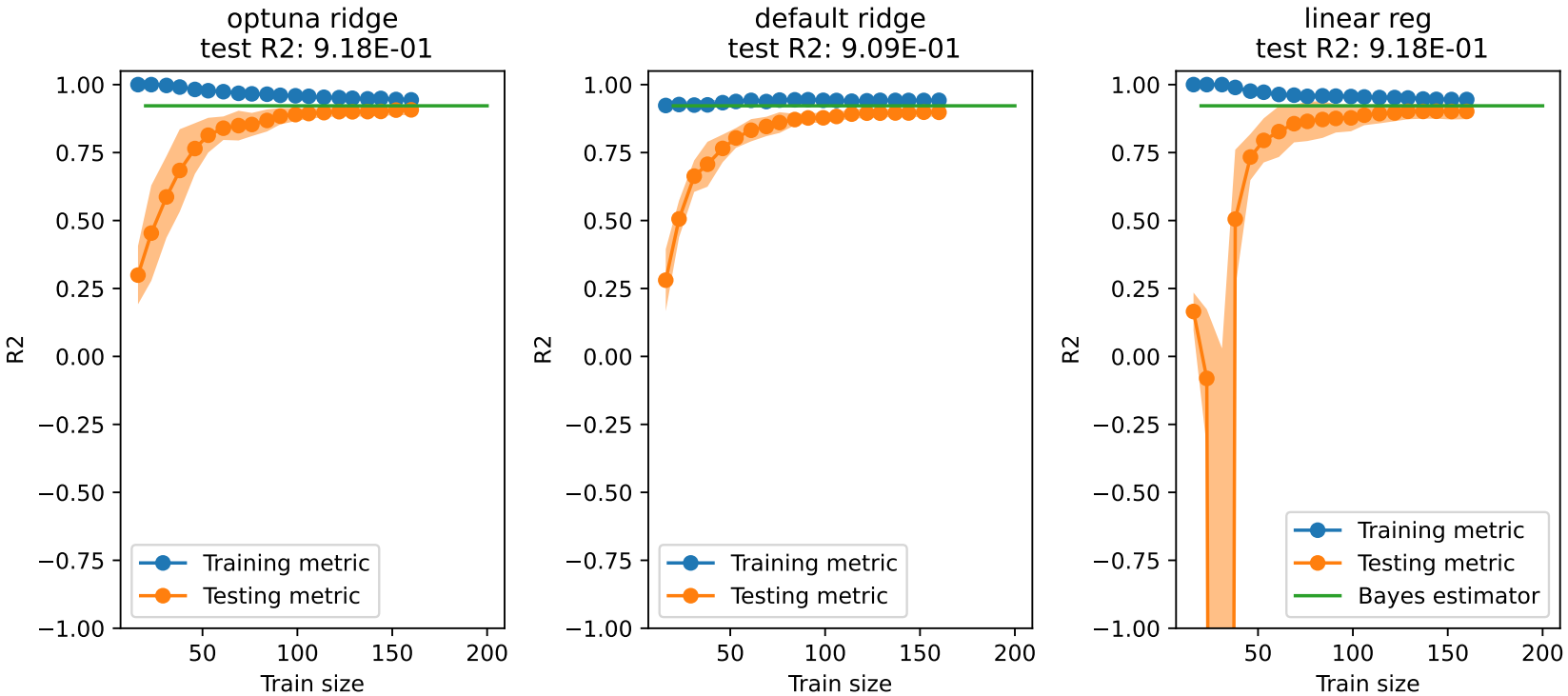
To choose λ_1 and λ_2 : cross validation.

In the following examples, the data are linear, with a sparse θ^*.

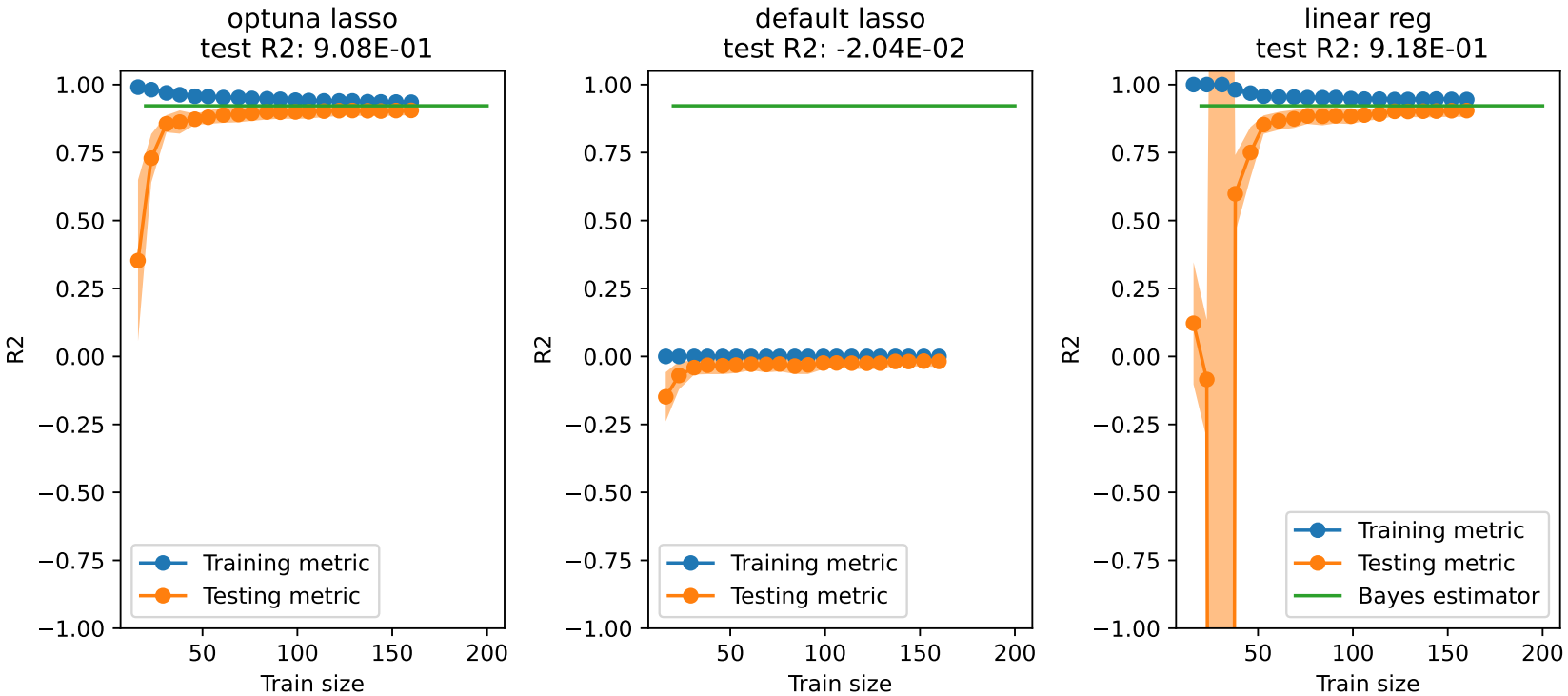
$$y_i = x^T \theta^* + \epsilon_i \tag{23}$$

We compare Ridge and Lasso, for several dimensions (n, d).

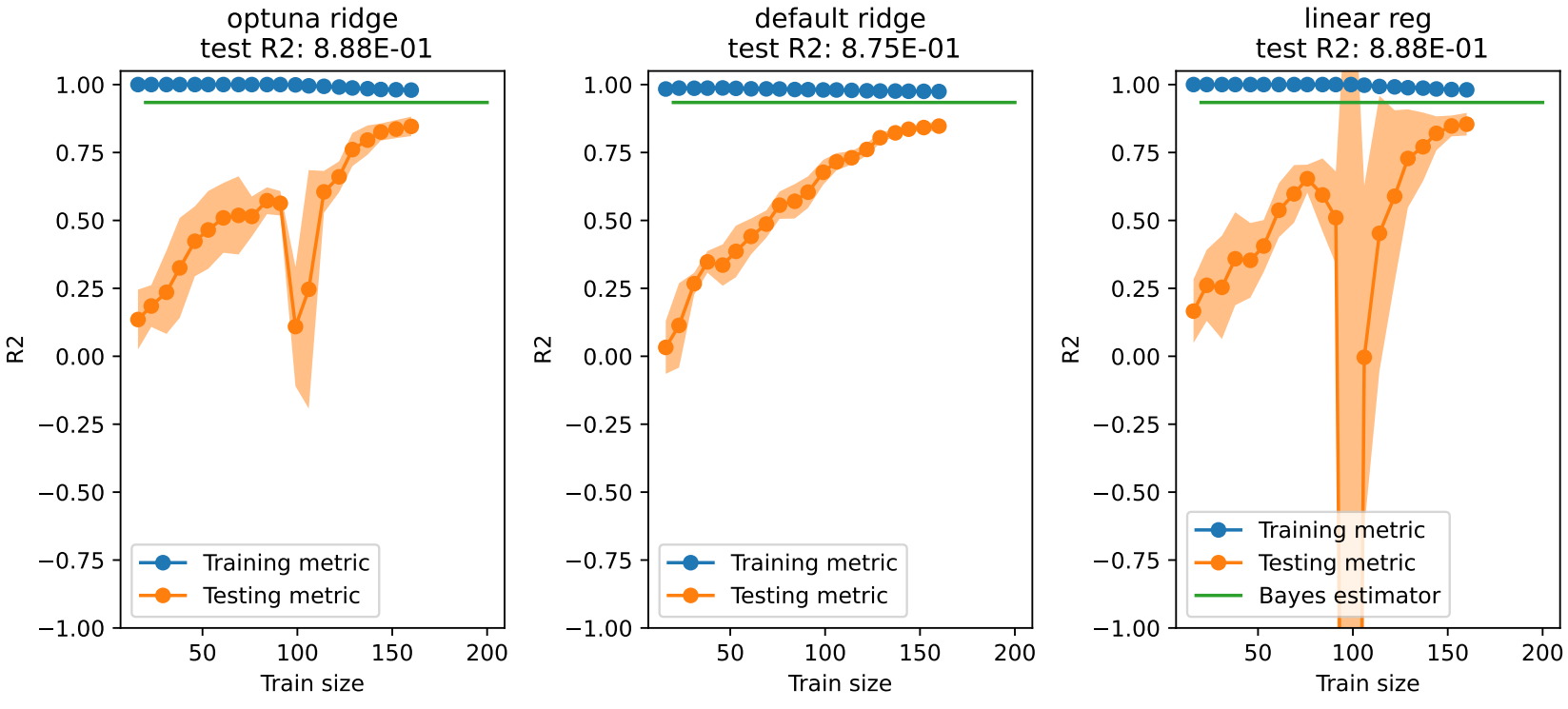
Learning curves ridge
Bayes risk: 4.000E-02
n=200, d=30
60 optuna trials
average time per trial: 4.53E-03s



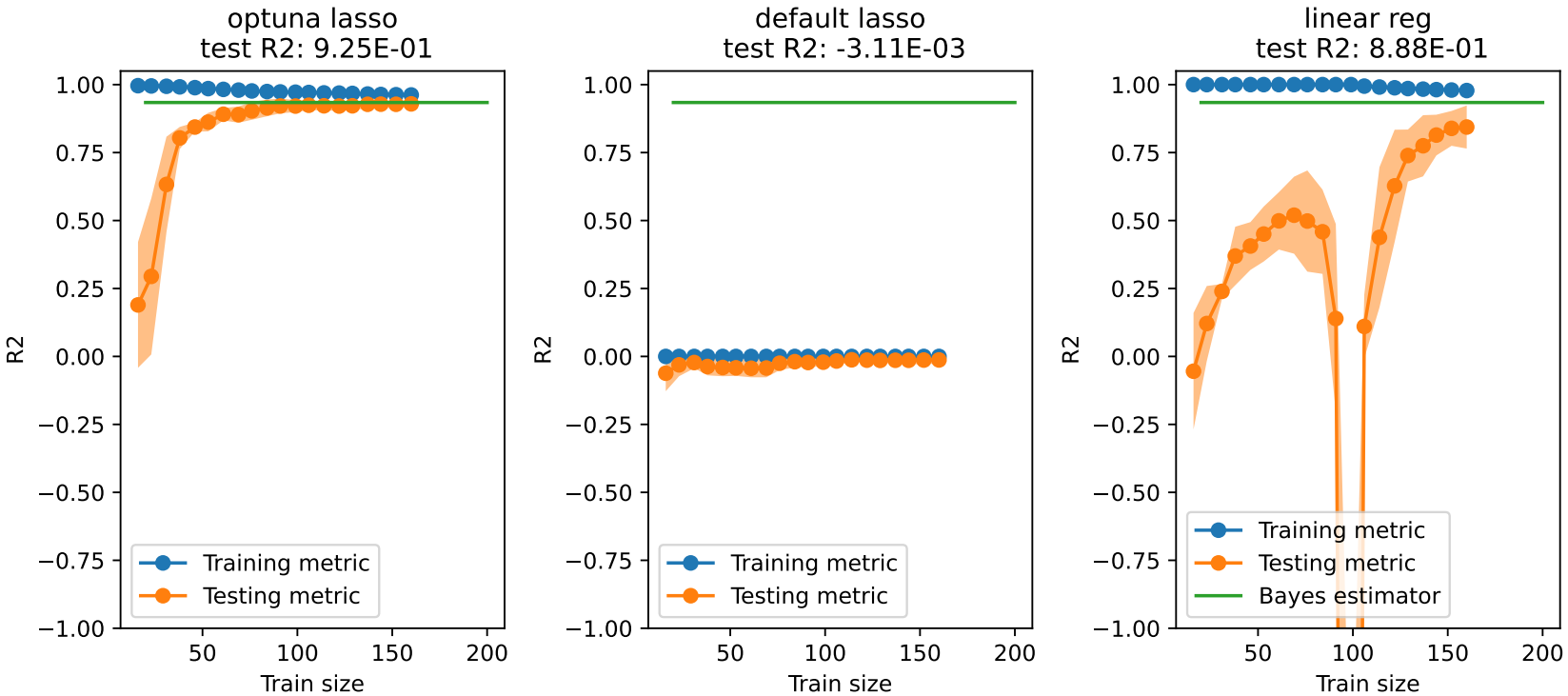
Learning curves lasso
Bayes risk: 4.000E-02
n=200, d=30
60 optuna trials
average time per trial: 4.71E-03s



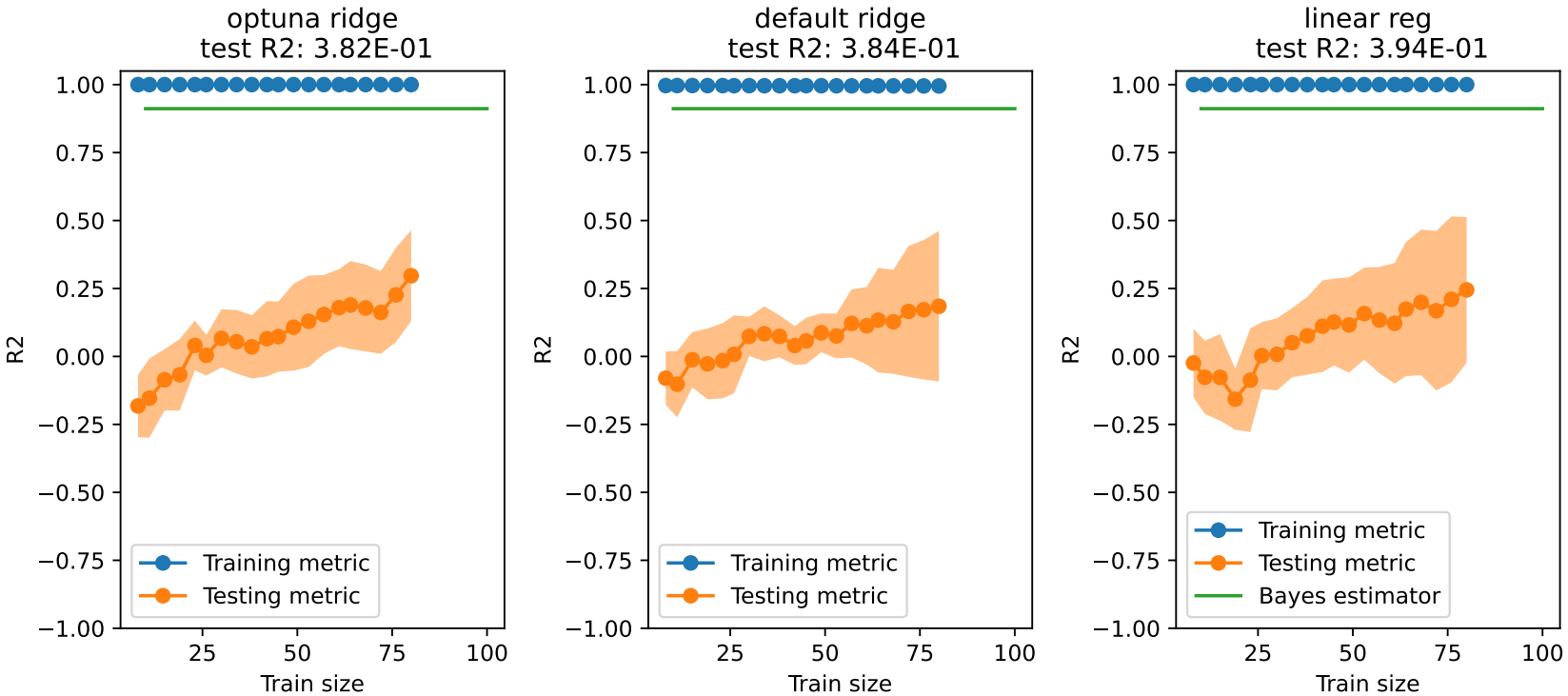
Learning curves ridge
Bayes risk: 4.000E-02
n=200, d=100
60 optuna trials
average time per trial: 1.61E+00s



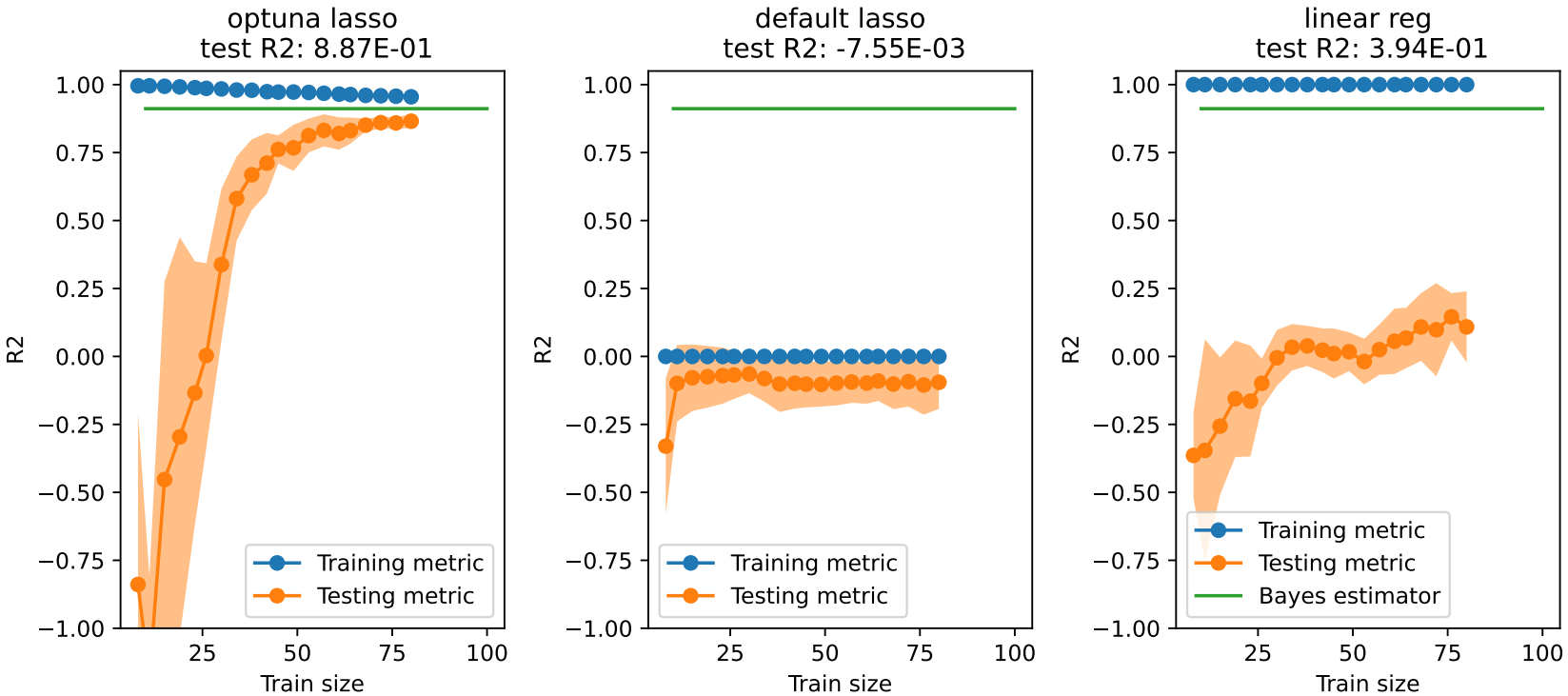
Learning curves lasso
Bayes risk: 4.000E-02
n=200, d=100
60 optuna trials
average time per trial: 1.54E-02s



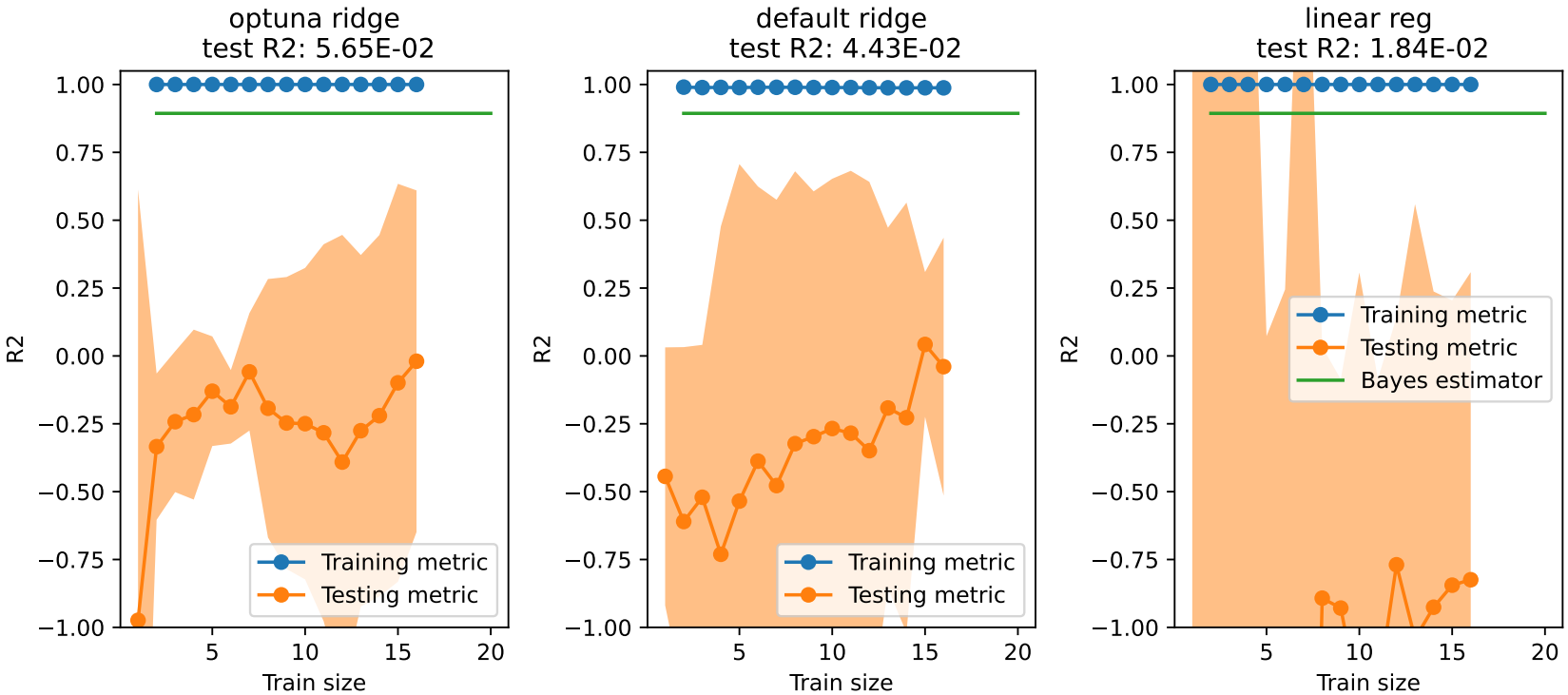
Learning curves ridge
Bayes risk: 4.000E-02
n=100, d=200
60 optuna trials
average time per trial: 1.24E+00s



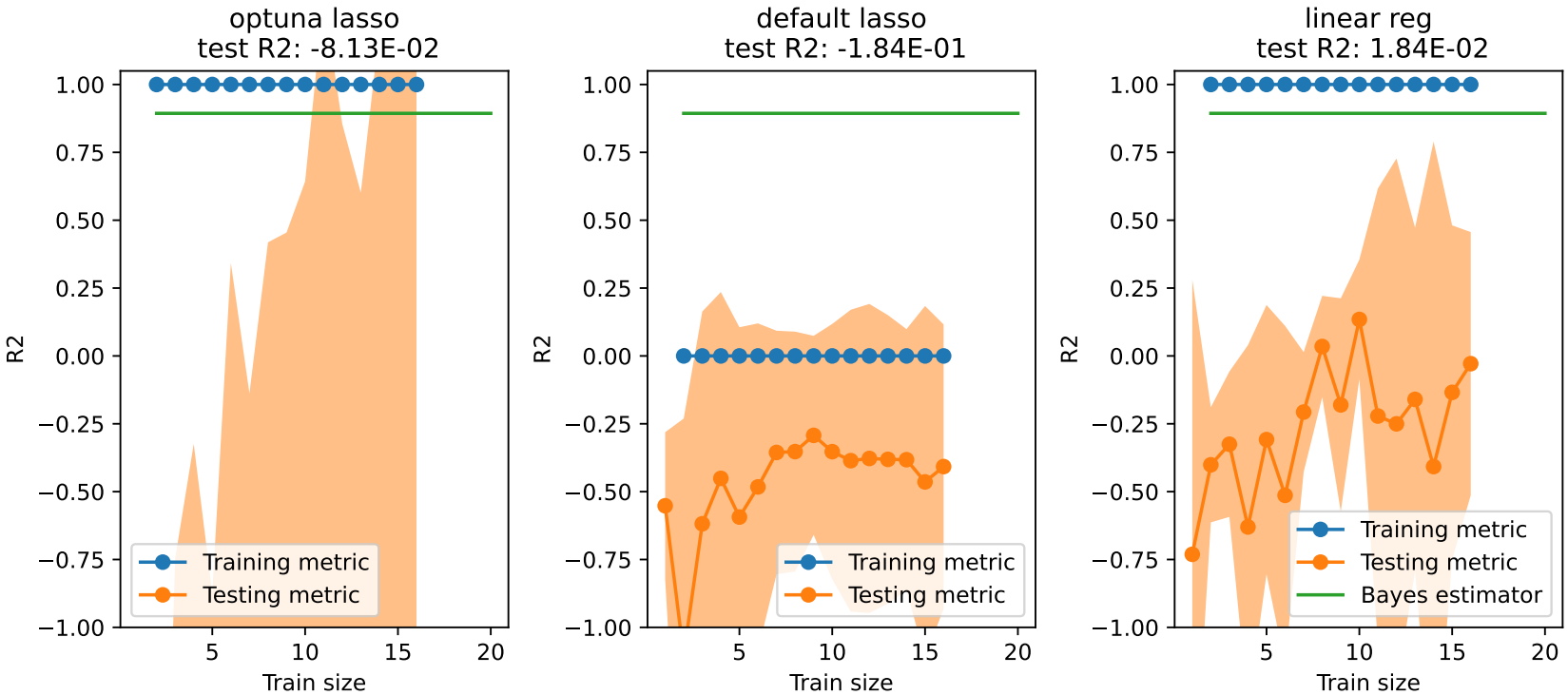
Learning curves lasso
Bayes risk: 4.000E-02
n=100, d=200
60 optuna trials
average time per trial: 1.77E-02s



Learning curves ridge
Bayes risk: 4.000E-02
n=20, d=100
60 optuna trials
average time per trial: 4.70E-03s



Learning curves lasso
Bayes risk: 4.000E-02
n=20, d=100
60 optuna trials
average time per trial: 8.92E-03s



Multiple objective optimization

A estimator might be considered good for several reasons :

- quality of the predictions
- short(er) optimization time
- small(er) computational ressources

Multiple-objective optimization

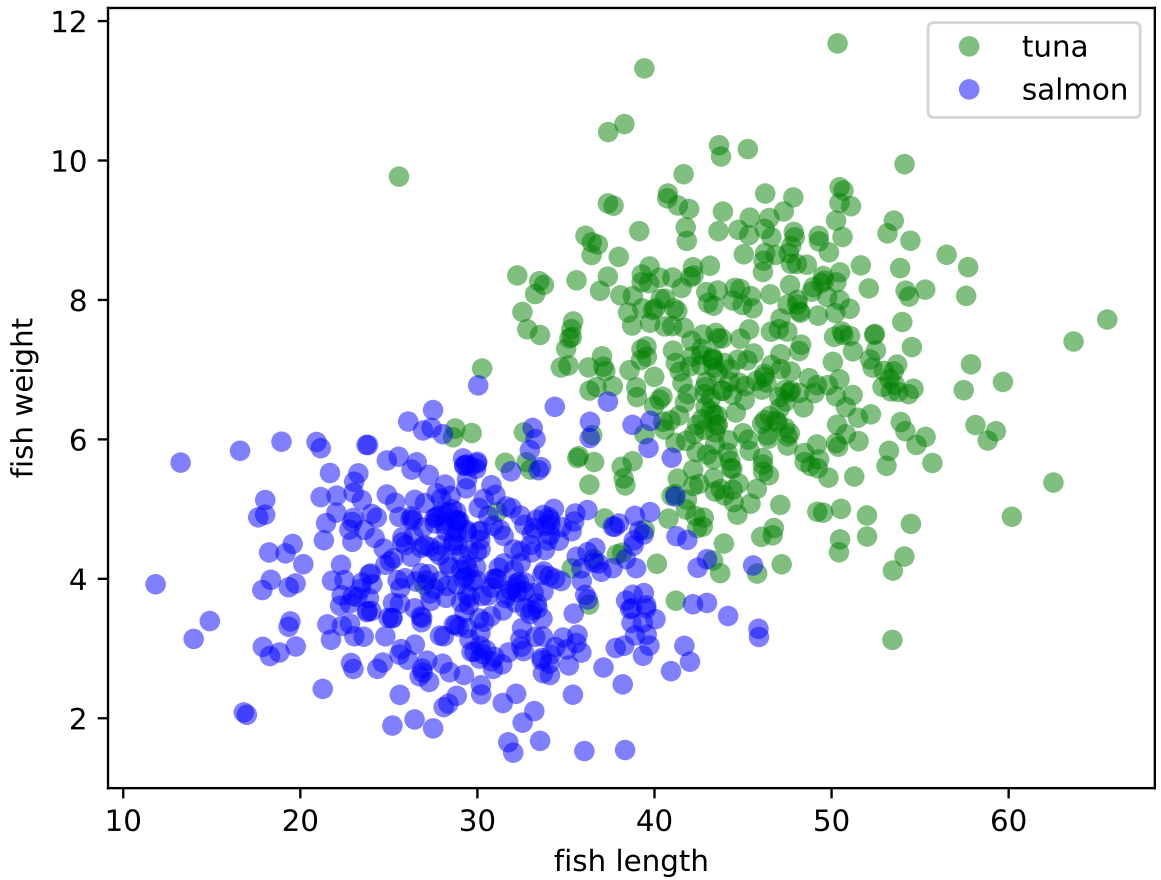
In optuna for instance, it is possible to do **mutliple objective** optimization.
https://en.wikipedia.org/wiki/Pareto_front
https://optuna.readthedocs.io/en/stable/tutorial/20_recipes/002_multi_objective.html
https://optuna.readthedocs.io/en/stable/reference/visualization/generated/optuna.visualization.plot_pareto_front.html

https://arxiv.org/pdf/2501.05515

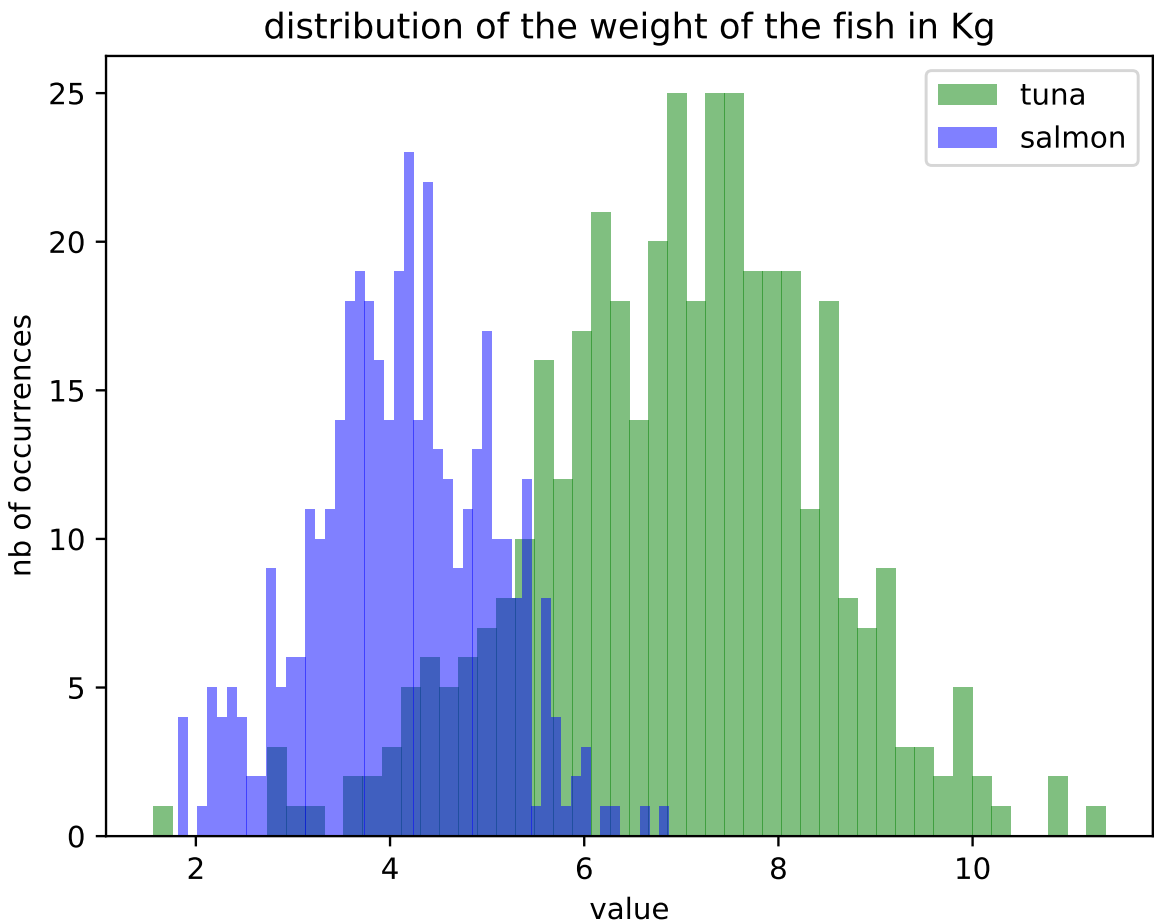
CART

- Segmentation method (partitionnement récursif), **binary splits.**
- First algorithm : **CART** (classification and regression tree, Breiman, 1984) [Breiman et al., 2017]

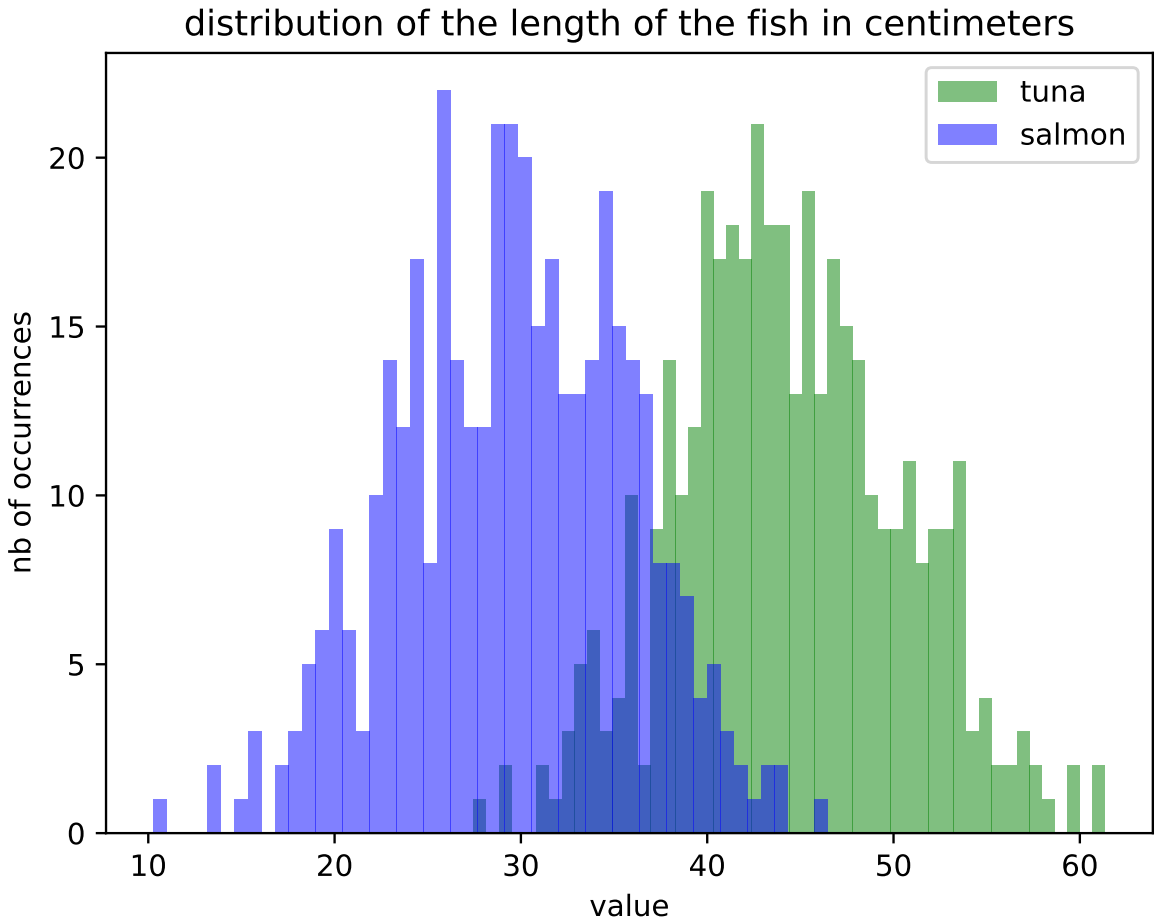
Example dataset : fishes



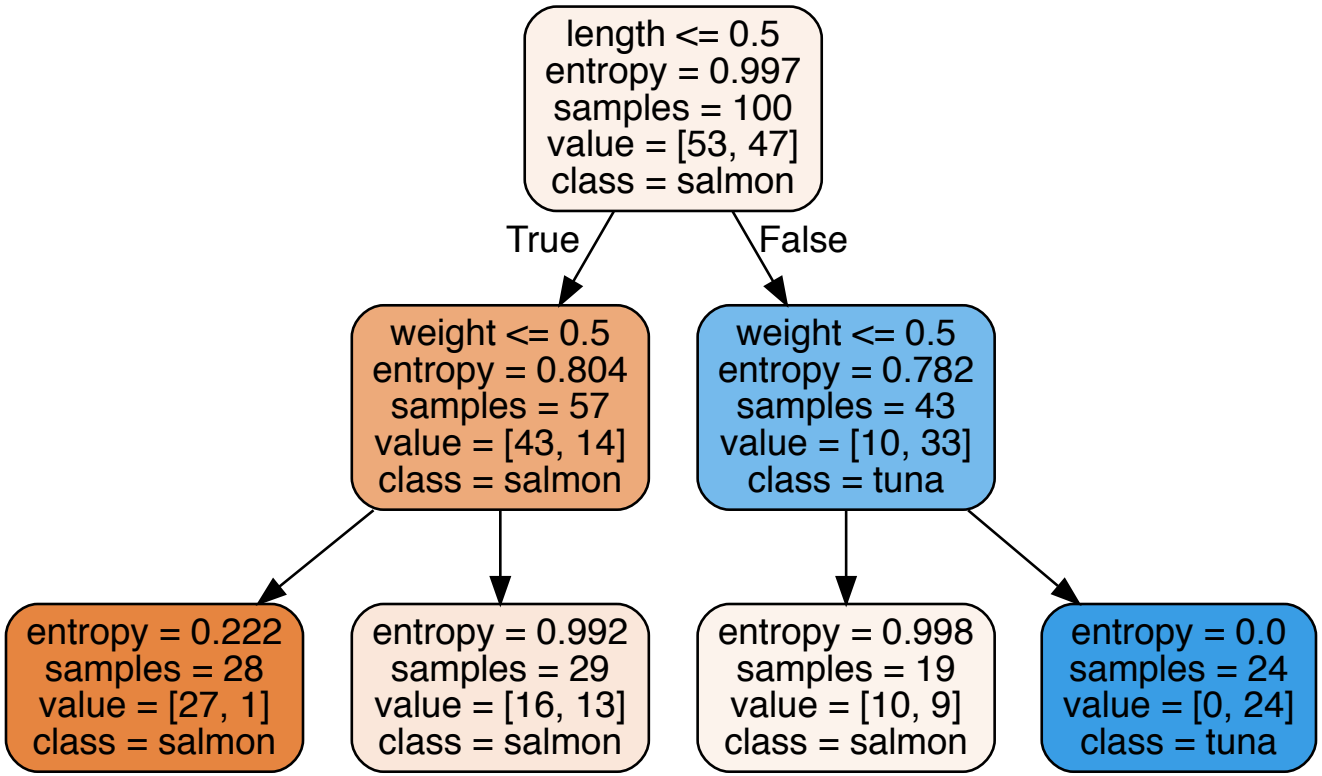
Fish dataset



Fish dataset

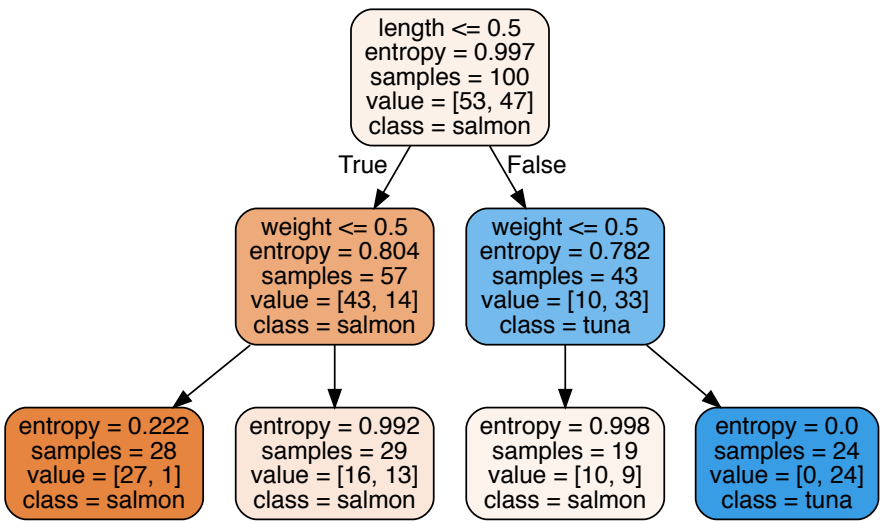


Example tree



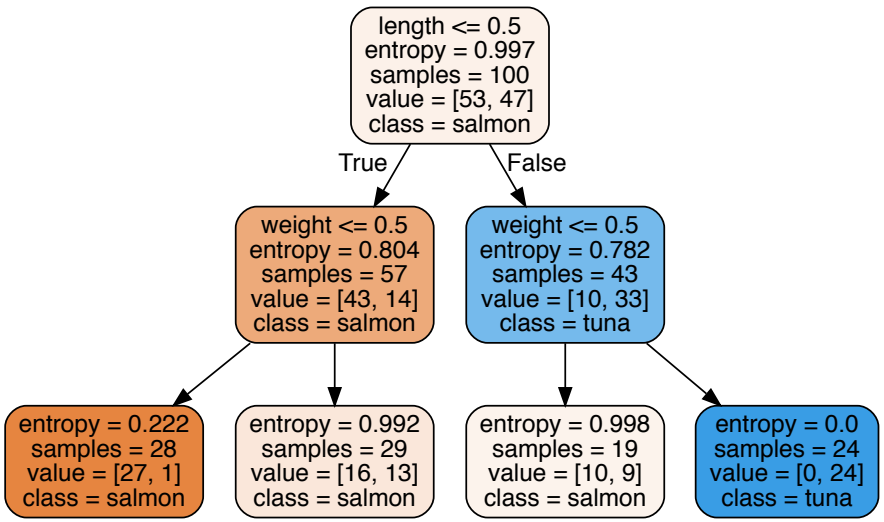
Splitting nodes

Each split should lead to more **homogeneous** nodes (in a sense that is to be formally defined)

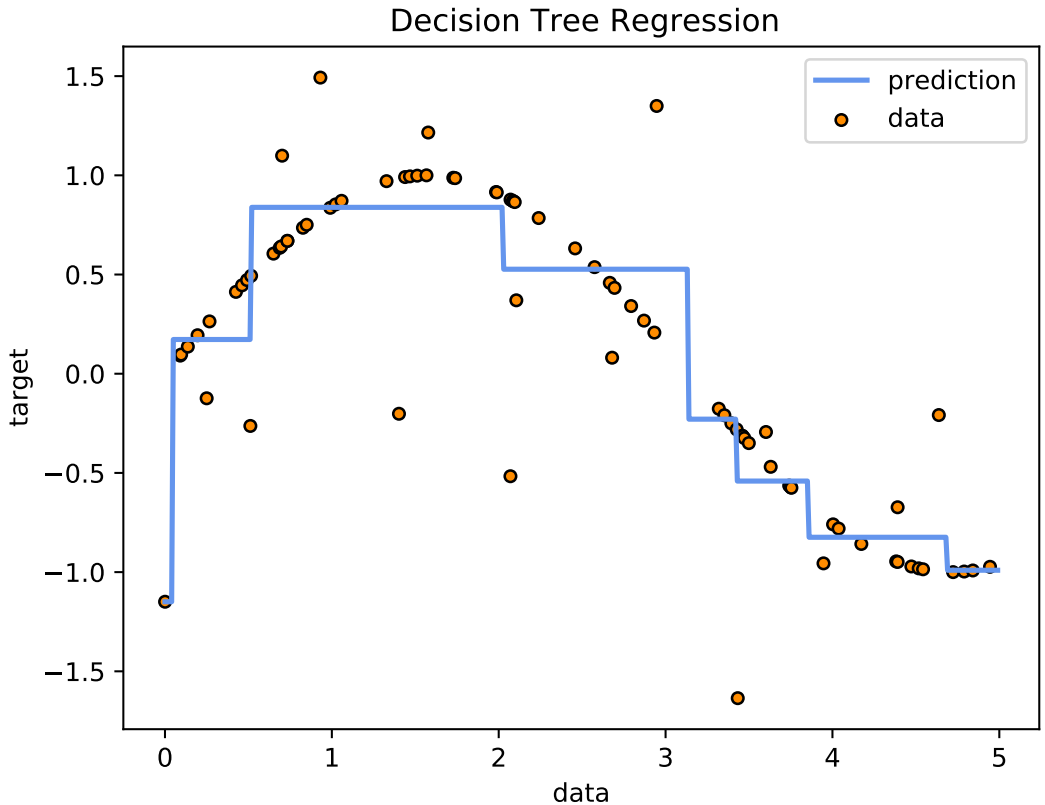


Splitting nodes

A leaf node corresponds to a predicted value.



Decision trees can be used for regression. In that case, the prediction is **piecewise constant**. It can be seen as a form of local averaging.



Construction of a tree

A split node corresponds to :

- a segmentation variable V
- a segmentation value / threshold if V is quantitative. If V is a class, it is a two-fold partitionning of the set of classes.

The construction of a tree requires a criterion to :

- choose the segmentation variable and value
- decide when a node is terminal (leaf node)
- associate a prediction value to all leaf nodes

Homogeneity

We want the homogeneity in the child nodes to be greater than in the parent node. Equivalently, we can define a non-negative heterogeneity function H that describes each node and that must be :

- 0 if the node is homogeneous (only one class or one output value is represented in this node)
- maximal if the values of y are uniformly distributed within the node.
- if $H(L)$ it the value of H for the left child node (and $H(R)$ for the right one), then the segmentation should minimize

$$H(L) + H(R) \tag{24}$$

Homogeneity criterion for regression

In the case of regression, $\mathcal{Y} = \mathbb{R}$ and the heterogeneity $H(n)$ of node n is its empirical variance :

$$H(n) = \frac{1}{|n|} \sum_{i \in n} (y_i - \bar{y}_n)^2 \tag{25}$$

Each y_i is a label for an input x_i that arrives in node n.

Homogeneity criterion for classification

In the case of classification, $\mathcal{Y} = [1, \ldots, L]$ and several criteria exist.

Homogeneity criterion for classification : entropy (information gain)

- We use the convention that $0 \log(0) = 0$
- p_n^l is the proportion of class l in node n.

The **entropy** writes :

$$H(n) = -\sum_{l=1}^{L} p_n^l \log(p_n^l) \tag{26}$$

Exercice 4 : What are the maximum and minimum values of the entropy ?

Homogeneity criterion for classification : Gini impurity

The Gini impurity writes :

$$H(n) = \sum_{l=1}^{L} p_n^l (1 - p_n^l) \tag{27}$$

Homogeneity criterion for classification : Gini impurity

The Gini impurity writes :

$$H(n) = \sum_{l=1}^{L} p_n^l (1 - p_n^l) \tag{28}$$

Exercice 5 : Interpret the meaning of the Gini impurity in terms of probabilities, assuming that we predict according to the proportions p_n^l.

Homogeneity criterion for classification : misclassification probability

$$H(n) = 1 - \max_{l}(p_n^l) \tag{29}$$

If we predict the most represented class in n, then this is the misclassification probability.

Prediction for a leaf node

- **regression** : predict the average value in the node
- **classification** : several possibilities
  - most represented class in the node
  - most probable class *a posteriori* if the *a priori* probabilities are known (Bayesian probabilities)
  - class that costs the less in case of missclassification

Pruning

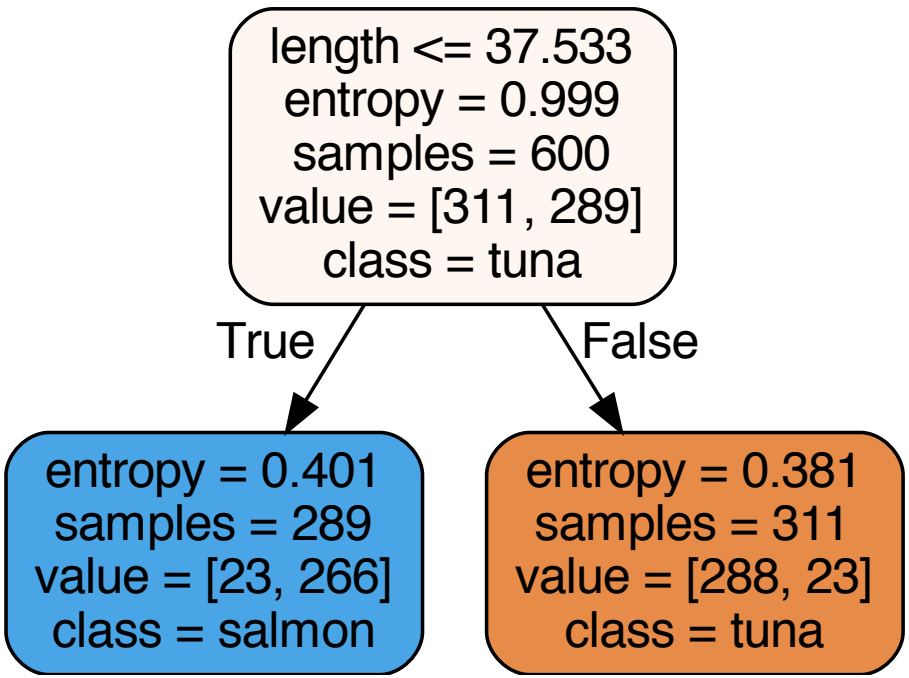
- If the segmentation is not restricted, the tree can fit the training dataset perfectly (overfitting).
- learning would then be highly dependent on the choice of the training dataset, leading to **instability.**
- To avoid it, **pruning** (élaguage) is used : it consists in restricting the size of the tree (and can be seen as an equivalent to regularization like in ridge regression or logistic regression).

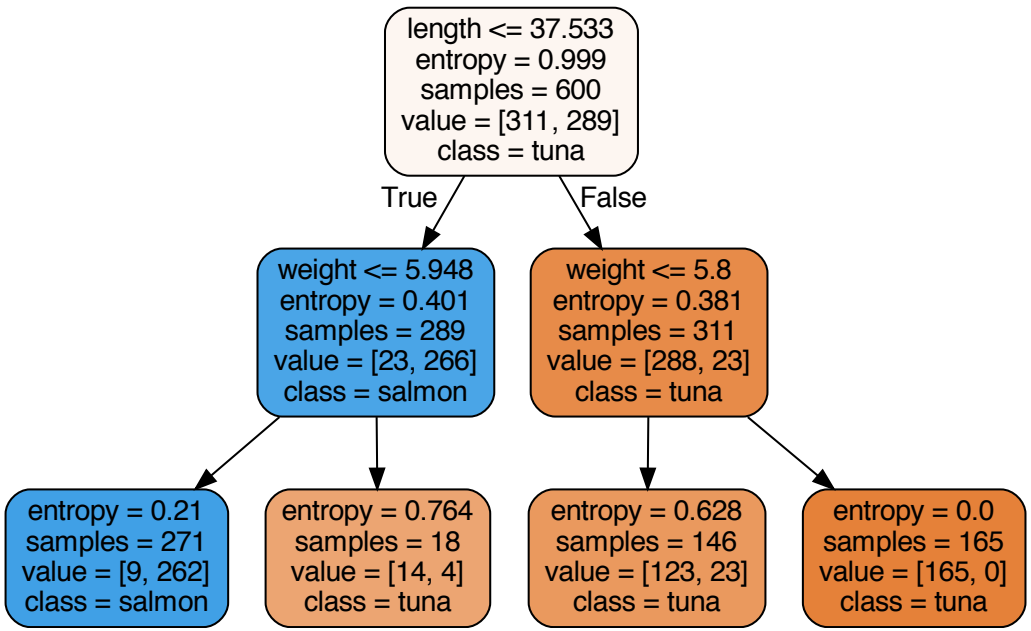
Pre-pruning by restricting divisions

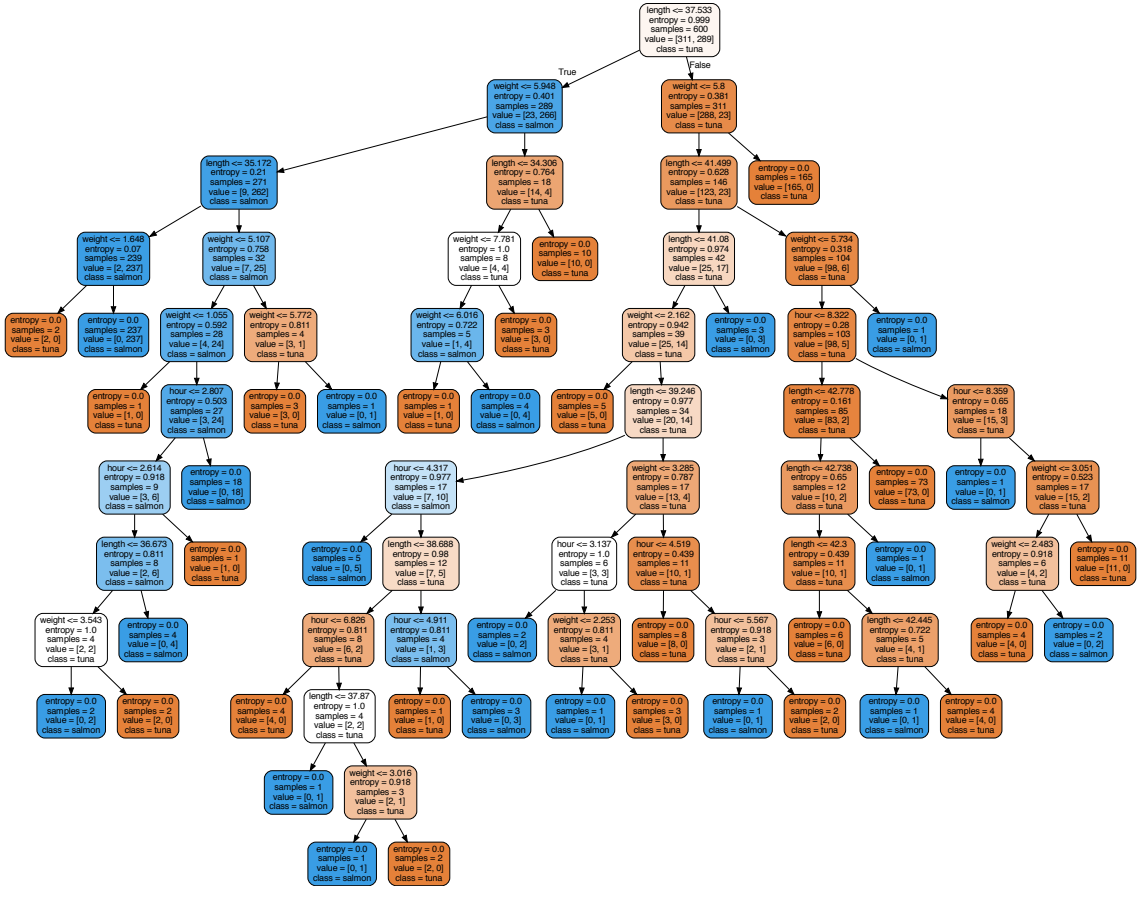
We can impose

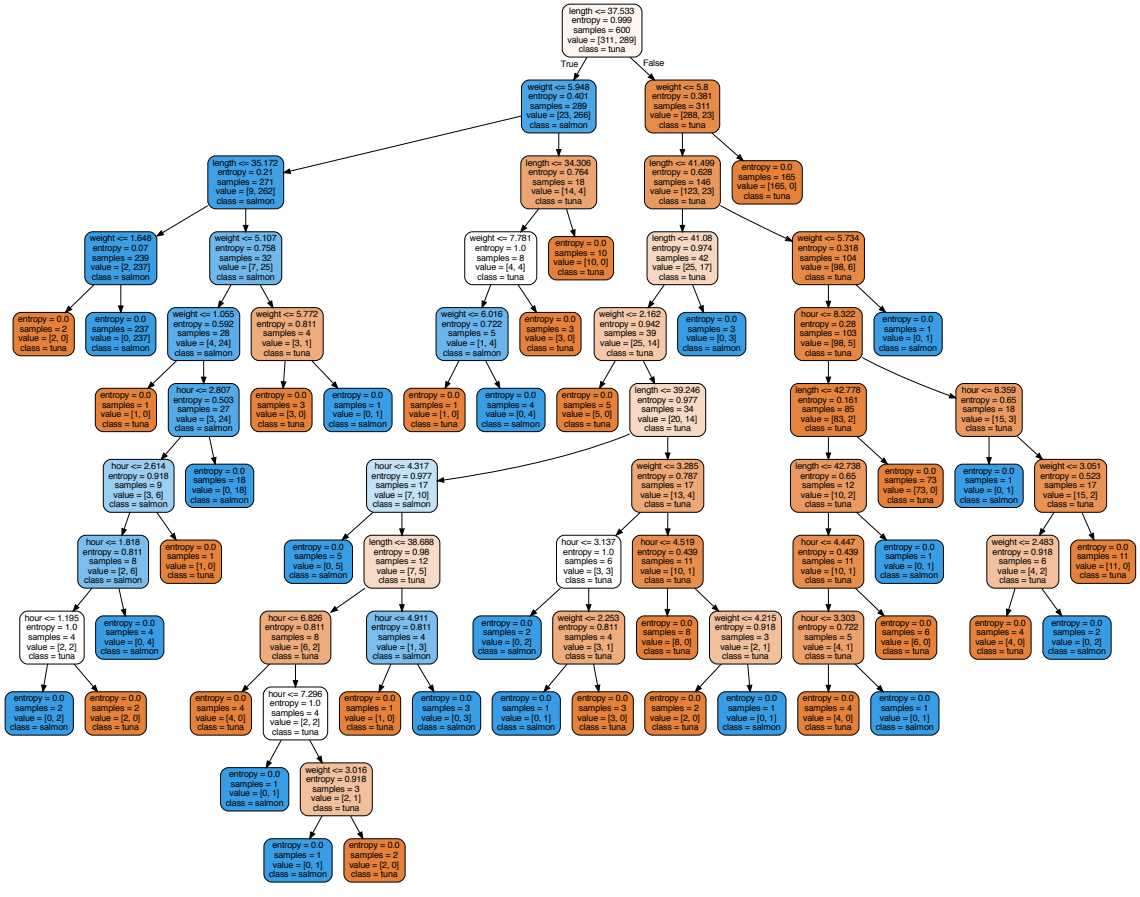
- minimum impurity decrease
- a minimum number of samples in a leaf node
- a minimum number of samples to authorize splitting a node

Simulation : fish dataset, max depth=1

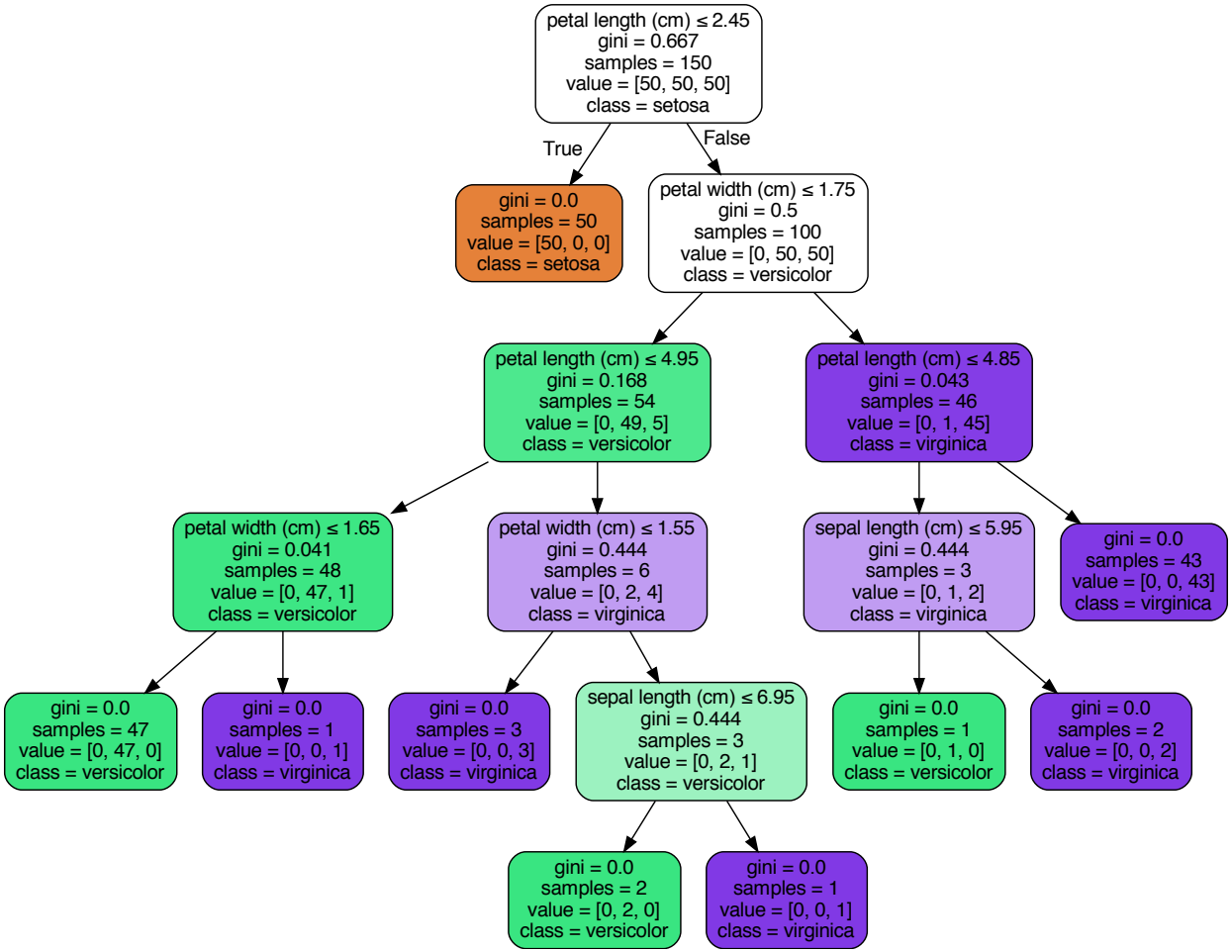



length <= 37.533
entropy = 0.999
samples = 600
value = [311, 289]
class = tuna
True
False
weight <= 5.948
entropy = 0.401
samples = 289
value = [23, 266]
class = salmon
weight <= 5.8
entropy = 0.381
samples = 311
value = [288, 23]
class = tuna
entropy = 0.21
samples = 271
value = [9, 262]
class = salmon
entropy = 0.764
samples = 18
value = [14, 4]
class = tuna
entropy = 0.628
samples = 146
value = [123, 23]
class = tuna
entropy = 0.0
samples = 165
value = [165, 0]
class = tuna


length <= 37.533
entropy = 0.999
samples = 600
value = [311, 289]
class = tuna
True
False
weight <= 5.948
entropy = 0.401
samples = 289
value = [23, 266]
class = salmon
weight <= 5.8
entropy = 0.381
samples = 311
value = [288, 23]
class = tuna
length <= 35.172
entropy = 0.21
samples = 271
value = [9, 262]
class = salmon
length <= 34.306
entropy = 0.764
samples = 18
value = [14, 4]
class = tuna
length <= 41.496
entropy = 0.628
samples = 146
value = [123, 23]
class = tuna
entropy = 0.0
samples = 165
value = [165, 0]
class = tuna
weight <= 1.648
entropy = 0.07
samples = 239
value = [2, 237]
class = salmon
weight <= 5.107
entropy = 0.758
samples = 32
value = [7, 25]
class = salmon
weight <= 7.781
entropy = 1.0
samples = 8
value = [4, 4]
class = tuna
entropy = 0.0
samples = 10
value = [10, 0]
class = tuna
length <= 41.08
entropy = 0.974
samples = 42
value = [25, 17]
class = tuna
weight <= 5.734
entropy = 0.318
samples = 104
value = [98, 6]
class = tuna
entropy = 0.0
samples = 2
value = [2, 0]
class = tuna
entropy = 0.0
samples = 237
value = [0, 237]
class = salmon
weight <= 1.055
entropy = 0.592
samples = 28
value = [4, 24]
class = salmon
weight <= 5.772
entropy = 0.811
samples = 4
value = [3, 1]
class = tuna
weight <= 6.016
entropy = 0.722
samples = 5
value = [1, 4]
class = salmon
entropy = 0.0
samples = 3
value = [3, 0]
class = tuna
weight <= 2.162
entropy = 0.942
samples = 39
value = [25, 14]
class = tuna
entropy = 0.0
samples = 3
value = [0, 3]
class = salmon
hour <= 8.322
entropy = 0.28
samples = 103
value = [98, 5]
class = tuna
entropy = 0.0
samples = 1
value = [0, 1]
class = salmon
entropy = 0.0
samples = 1
value = [1, 0]
class = tuna
hour <= 2.807
entropy = 0.503
samples = 27
value = [3, 24]
class = salmon
entropy = 0.0
samples = 3
value = [3, 0]
class = tuna
entropy = 0.0
samples = 1
value = [0, 1]
class = salmon
entropy = 0.0
samples = 1
value = [1, 0]
class = tuna
entropy = 0.0
samples = 4
value = [0, 4]
class = salmon
entropy = 0.0
samples = 5
value = [5, 0]
class = tuna
length <= 39.246
entropy = 0.977
samples = 34
value = [20, 14]
class = tuna
length <= 42.778
entropy = 0.161
samples = 85
value = [83, 2]
class = tuna
hour <= 8.359
entropy = 0.65
samples = 18
value = [15, 3]
class = tuna
hour <= 2.614
entropy = 0.918
samples = 9
value = [3, 6]
class = salmon
entropy = 0.0
samples = 18
value = [0, 18]
class = salmon
hour <= 4.317
entropy = 0.977
samples = 17
value = [7, 10]
class = salmon
weight <= 3.285
entropy = 0.787
samples = 17
value = [13, 4]
class = tuna
length <= 42.738
entropy = 0.65
samples = 12
value = [10, 2]
class = tuna
entropy = 0.0
samples = 73
value = [73, 0]
class = tuna
entropy = 0.0
samples = 1
value = [0, 1]
class = salmon
weight <= 3.051
entropy = 0.523
samples = 17
value = [15, 2]
class = tuna
length <= 36.673
entropy = 0.811
samples = 8
value = [2, 6]
class = salmon
entropy = 0.0
samples = 1
value = [1, 0]
class = tuna
entropy = 0.0
samples = 5
value = [0, 5]
class = salmon
length <= 38.688
entropy = 0.98
samples = 12
value = [7, 5]
class = tuna
hour <= 3.137
entropy = 1.0
samples = 6
value = [3, 3]
class = tuna
hour <= 4.519
entropy = 0.439
samples = 11
value = [10, 1]
class = tuna
length <= 42.3
entropy = 0.439
samples = 11
value = [10, 1]
class = tuna
entropy = 0.0
samples = 1
value = [0, 1]
class = salmon
weight <= 2.483
entropy = 0.918
samples = 6
value = [4, 2]
class = tuna
entropy = 0.0
samples = 11
value = [11, 0]
class = tuna
weight <= 3.543
entropy = 1.0
samples = 4
value = [2, 2]
class = tuna
entropy = 0.0
samples = 4
value = [0, 4]
class = salmon
hour <= 6.826
entropy = 0.811
samples = 8
value = [6, 2]
class = tuna
hour <= 4.911
entropy = 0.811
samples = 4
value = [1, 3]
class = salmon
entropy = 0.0
samples = 2
value = [0, 2]
class = salmon
weight <= 2.253
entropy = 0.811
samples = 4
value = [3, 1]
class = tuna
entropy = 0.0
samples = 8
value = [8, 0]
class = tuna
hour <= 5.567
entropy = 0.918
samples = 3
value = [2, 1]
class = tuna
entropy = 0.0
samples = 6
value = [6, 0]
class = tuna
length <= 42.445
entropy = 0.722
samples = 5
value = [4, 1]
class = tuna
entropy = 0.0
samples = 4
value = [4, 0]
class = tuna
entropy = 0.0
samples = 2
value = [0, 2]
class = salmon
entropy = 0.0
samples = 2
value = [2, 0]
class = tuna
entropy = 0.0
samples = 4
value = [4, 0]
class = tuna
length <= 37.87
entropy = 1.0
samples = 4
value = [2, 2]
class = tuna
entropy = 0.0
samples = 1
value = [1, 0]
class = tuna
entropy = 0.0
samples = 3
value = [0, 3]
class = salmon
entropy = 0.0
samples = 1
value = [1, 0]
class = salmon
entropy = 0.0
samples = 3
value = [3, 0]
class = tuna
entropy = 0.0
samples = 1
value = [0, 1]
class = salmon
entropy = 0.0
samples = 2
value = [2, 0]
class = tuna
entropy = 0.0
samples = 1
value = [0, 1]
class = salmon
weight <= 3.016
entropy = 0.918
samples = 3
value = [2, 1]
class = tuna
entropy = 0.0
samples = 1
value = [0, 1]
class = salmon
entropy = 0.0
samples = 2
value = [2, 0]
class = tuna


length <= 37.533
entropy = 0.999
samples = 600
value = [311, 289]
class = tuna
True
False
weight <= 5.948
entropy = 0.401
samples = 289
value = [23, 266]
class = salmon
weight <= 5.8
entropy = 0.381
samples = 311
value = [288, 23]
class = tuna
length <= 35.172
entropy = 0.21
samples = 271
value = [9, 262]
class = salmon
length <= 34.306
entropy = 0.754
samples = 18
value = [14, 4]
class = tuna
length <= 41.499
entropy = 0.628
samples = 146
value = [123, 23]
class = tuna
entropy = 0.0
samples = 165
value = [165, 0]
class = tuna
weight <= 1.648
entropy = 0.07
samples = 239
value = [2, 237]
class = salmon
weight <= 5.107
entropy = 0.758
samples = 32
value = [7, 25]
class = salmon
weight <= 7.781
entropy = 1.0
samples = 8
value = [4, 4]
class = tuna
entropy = 0.0
samples = 10
value = [10, 0]
class = tuna
length <= 41.08
entropy = 0.974
samples = 42
value = [25, 17]
class = tuna
weight <= 5.734
entropy = 0.318
samples = 104
value = [98, 6]
class = tuna
entropy = 0.0
samples = 2
value = [2, 0]
class = tuna
entropy = 0.0
samples = 237
value = [0, 237]
class = salmon
weight <= 1.055
entropy = 0.592
samples = 28
value = [4, 24]
class = salmon
weight <= 5.772
entropy = 0.811
samples = 4
value = [3, 1]
class = tuna
weight <= 6.016
entropy = 0.722
samples = 5
value = [1, 4]
class = salmon
entropy = 0.0
samples = 3
value = [3, 0]
class = tuna
weight <= 2.162
entropy = 0.942
samples = 39
value = [25, 14]
class = tuna
entropy = 0.0
samples = 3
value = [0, 3]
class = salmon
hour <= 8.322
entropy = 0.28
samples = 103
value = [98, 5]
class = tuna
entropy = 0.0
samples = 1
value = [0, 1]
class = salmon
entropy = 0.0
samples = 1
value = [1, 0]
class = tuna
hour <= 2.807
entropy = 0.503
samples = 27
value = [3, 24]
class = salmon
entropy = 0.0
samples = 3
value = [3, 0]
class = tuna
entropy = 0.0
samples = 1
value = [0, 1]
class = salmon
entropy = 0.0
samples = 1
value = [1, 0]
class = tuna
entropy = 0.0
samples = 4
value = [0, 4]
class = salmon
entropy = 0.0
samples = 5
value = [5, 0]
class = tuna
length <= 39.246
entropy = 0.977
samples = 34
value = [20, 14]
class = tuna
length <= 42.778
entropy = 0.161
samples = 85
value = [83, 2]
class = tuna
hour <= 8.359
entropy = 0.65
samples = 18
value = [15, 3]
class = tuna
hour <= 2.614
entropy = 0.918
samples = 9
value = [3, 6]
class = salmon
entropy = 0.0
samples = 18
value = [0, 18]
class = salmon
hour <= 4.317
entropy = 0.977
samples = 17
value = [7, 10]
class = salmon
weight <= 3.285
entropy = 0.787
samples = 17
value = [13, 4]
class = tuna
length <= 42.738
entropy = 0.65
samples = 12
value = [10, 2]
class = tuna
entropy = 0.0
samples = 73
value = [73, 0]
class = tuna
entropy = 0.0
samples = 1
value = [0, 1]
class = salmon
weight <= 3.051
entropy = 0.523
samples = 17
value = [15, 2]
class = tuna
hour <= 1.818
entropy = 0.811
samples = 8
value = [2, 6]
class = salmon
entropy = 0.0
samples = 1
value = [1, 0]
class = tuna
entropy = 0.0
samples = 5
value = [0, 5]
class = salmon
length <= 38.688
entropy = 0.98
samples = 12
value = [7, 5]
class = tuna
hour <= 3.137
entropy = 1.0
samples = 6
value = [3, 3]
class = tuna
hour <= 4.519
entropy = 0.439
samples = 11
value = [10, 1]
class = tuna
hour <= 4.447
entropy = 0.439
samples = 11
value = [10, 1]
class = tuna
entropy = 0.0
samples = 1
value = [0, 1]
class = salmon
weight <= 2.483
entropy = 0.918
samples = 6
value = [4, 2]
class = tuna
entropy = 0.0
samples = 11
value = [11, 0]
class = tuna
hour <= 1.195
entropy = 1.0
samples = 4
value = [2, 2]
class = tuna
entropy = 0.0
samples = 4
value = [0, 4]
class = salmon
hour <= 6.826
entropy = 0.811
samples = 8
value = [6, 2]
class = tuna
hour <= 4.911
entropy = 0.811
samples = 4
value = [1, 3]
class = salmon
entropy = 0.0
samples = 2
value = [0, 2]
class = salmon
weight <= 2.253
entropy = 0.811
samples = 4
value = [3, 1]
class = tuna
entropy = 0.0
samples = 8
value = [8, 0]
class = tuna
weight <= 4.215
entropy = 0.918
samples = 3
value = [2, 1]
class = tuna
hour <= 3.303
entropy = 0.722
samples = 5
value = [4, 1]
class = tuna
entropy = 0.0
samples = 6
value = [6, 0]
class = tuna
entropy = 0.0
samples = 4
value = [4, 0]
class = tuna
entropy = 0.0
samples = 2
value = [0, 2]
class = salmon
entropy = 0.0
samples = 2
value = [2, 0]
class = salmon
entropy = 0.0
samples = 4
value = [4, 0]
class = tuna
hour <= 7.296
entropy = 1.0
samples = 4
value = [2, 2]
class = tuna
entropy = 0.0
samples = 1
value = [1, 0]
class = tuna
entropy = 0.0
samples = 3
value = [0, 3]
class = salmon
entropy = 0.0
samples = 1
value = [1, 0]
class = tuna
entropy = 0.0
samples = 3
value = [3, 0]
class = salmon
entropy = 0.0
samples = 2
value = [2, 0]
class = tuna
entropy = 0.0
samples = 1
value = [0, 1]
class = tuna
entropy = 0.0
samples = 1
value = [0, 1]
class = salmon
weight <= 3.016
entropy = 0.918
samples = 3
value = [2, 1]
class = tuna
entropy = 0.0
samples = 1
value = [0, 1]
class = salmon
entropy = 0.0
samples = 2
value = [2, 0]
class = tuna

Overfitting : Iris dataset, max depth=34



Avoiding overfitting : Iris dataset

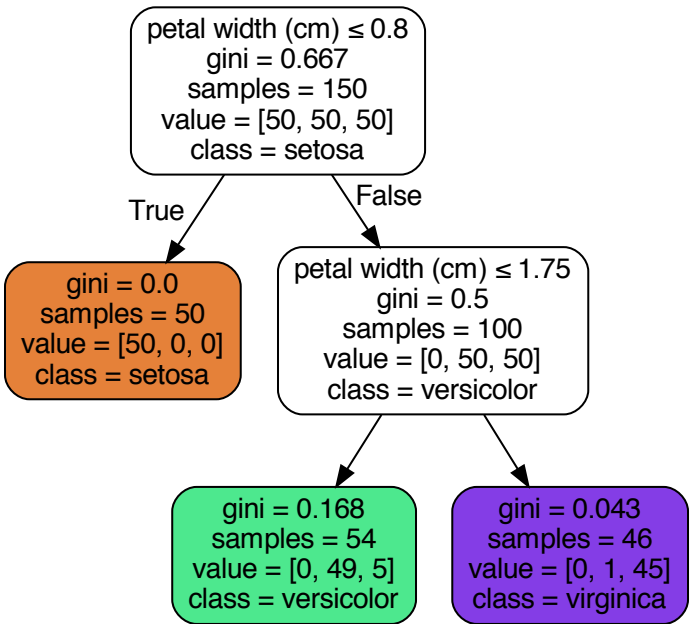


Figure – Avoid overfitting with pre-pruning. In this case, impose a minimum impurity decrease of 0.1

Post-pruning

It is also possible to prune after the construction of a large tree.

Remarks on decision trees I

- Decision trees do not require hypotheses on the distribution of de features.
- Feature selection is integrated in the algorithm, so they might be relevant in a situation with many features (variables explicatives)
- A segmentation is invariant to a monotonic change in a feature. It is thus robust to outliers or to very asymetrical distributions.
- Decision trees allow a straightforward **interpretation** of the model.

Remarks on decision trees II

- Decision trees are greedy : they might find a local minimum
- they are hierarchical : a bad choice in a segmentation at the top of the tree propagates in the whole tree
- Hence their instability and sensibility to the choice of the training set.

Ensemble learning

- *Bagging* and *boosting* are methods that combine estimators (in parallel or sequentially)
- they are often applied to decision trees
- they reach state-of-the-art performance in several supervised learning tasks [Fernández-Delgado et al., 2014]
- they are an active area of research (both theoretically and for practical applications).

`https://scikit-learn.org/stable/modules/ensemble.html`

Aggregating to reduce the variance

- usual setup : input space $\mathcal{X}$, output space $\mathcal{Y}$.
- we note z_b a dataset sampled from the unknown distribution ρ. We sample b different datasets, $b \in [1, B]$,

$$z_b = \{(x_{1b}, y_{1b}), \dots (x_{nb}, y_{nb})\} \tag{30}$$

- we note $\hat{f_{z_b}}$ the estimator obtained after learning with z_b.

Aggregate to reduce the variance

- we note z_b a dataset sampled from the unknown distribution ρ. If we sample b different datasets, $b \in [1, B]$,

$$z_b = \{(x_{1b}, y_{1b}), \dots (x_{nb}, y_{nb})\} \tag{31}$$

- we note $\hat{f}_{z_b}$ the estimator obtained after learning with z_b.

Aggregating consists in using as an estimator :

- for regression

$$\hat{f}_B = \frac{1}{B} \sum_{b=1}^{B} \hat{f}_{z_b} \tag{32}$$

- for classification

$$\hat{f}_B(x) = \arg\max_j |\{b, \hat{f}_{z_b}(x) = j\}| \tag{33}$$

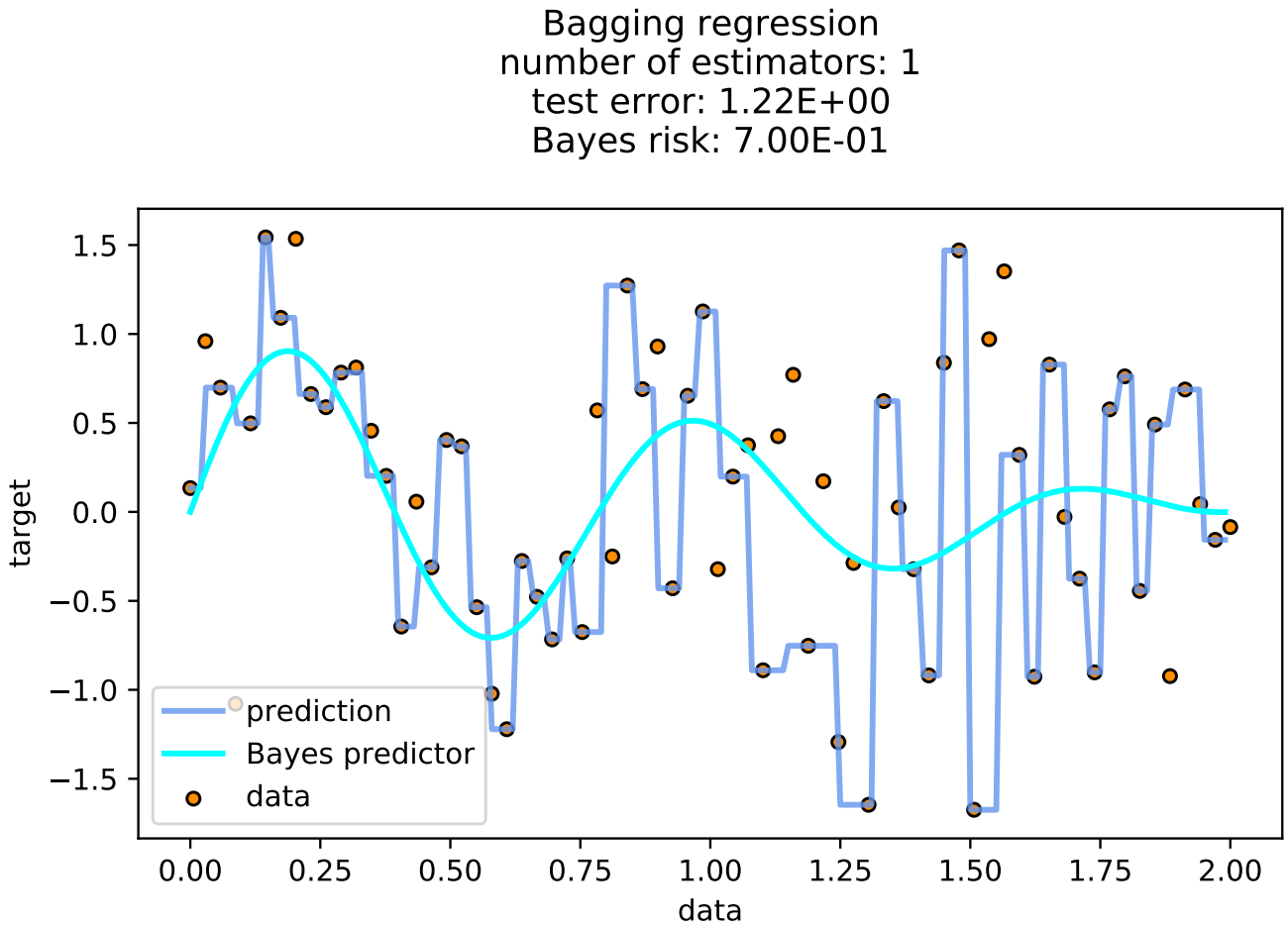
Bootstrapping and bagging

If B is large, it is not possible to sample B independent datasets with n samples, from a finite dataset.
Bootstrapping consists in sampling B times a sample dataset with n elements **with replacement**.
Bagging is the combination of bootstrapping and aggregating.

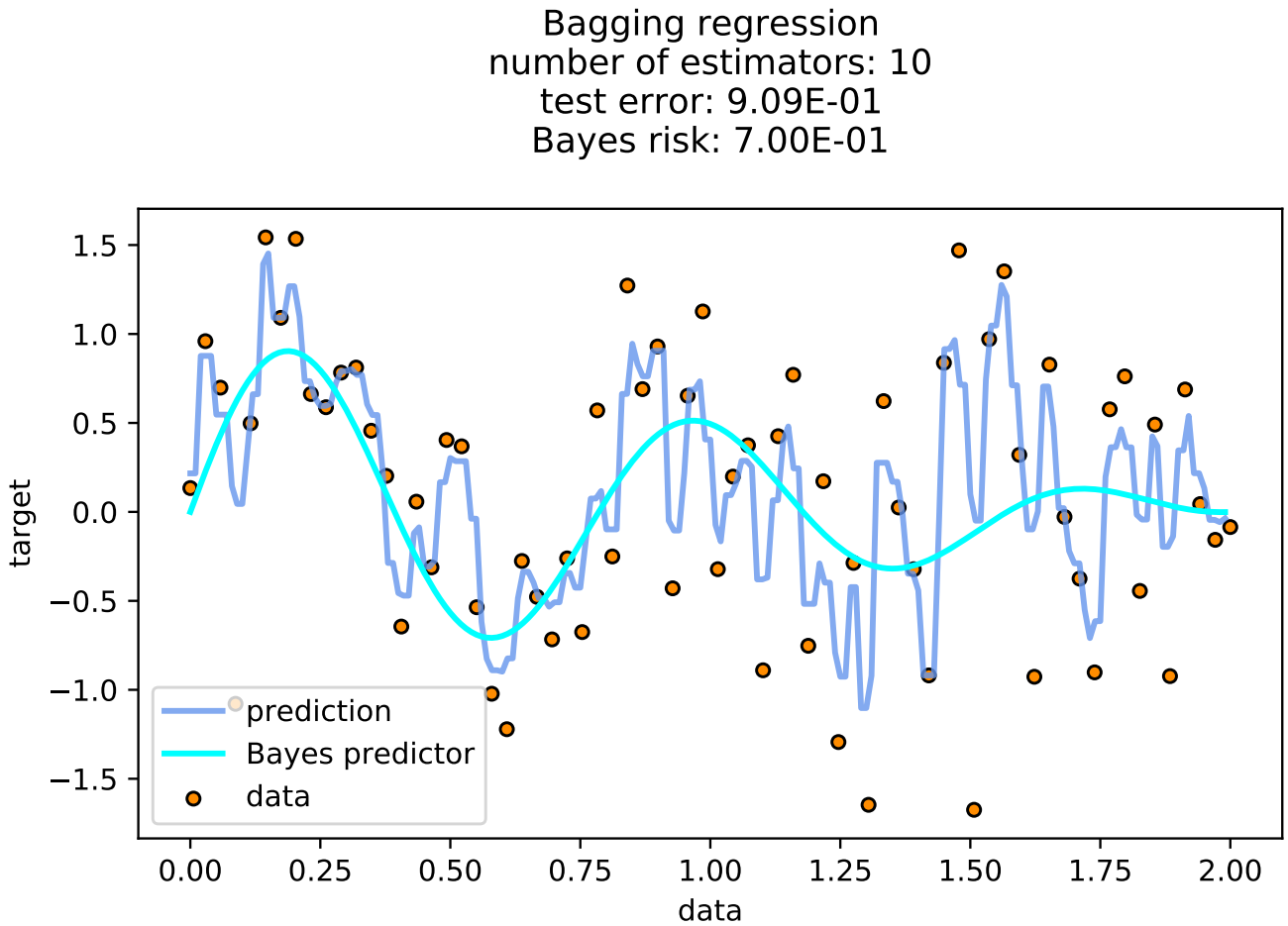
Out-of-bag error

Vocabulary : for an observation (x_i, y_i), the **out-of-bag error** is the mean error on this observation among the estimators that were trained on a bootstrap dataset **not** containing it.
It can be seen as an equivalent to the test error, the difference being that when doing bootstrapping, there is not a single train set and a single test set.

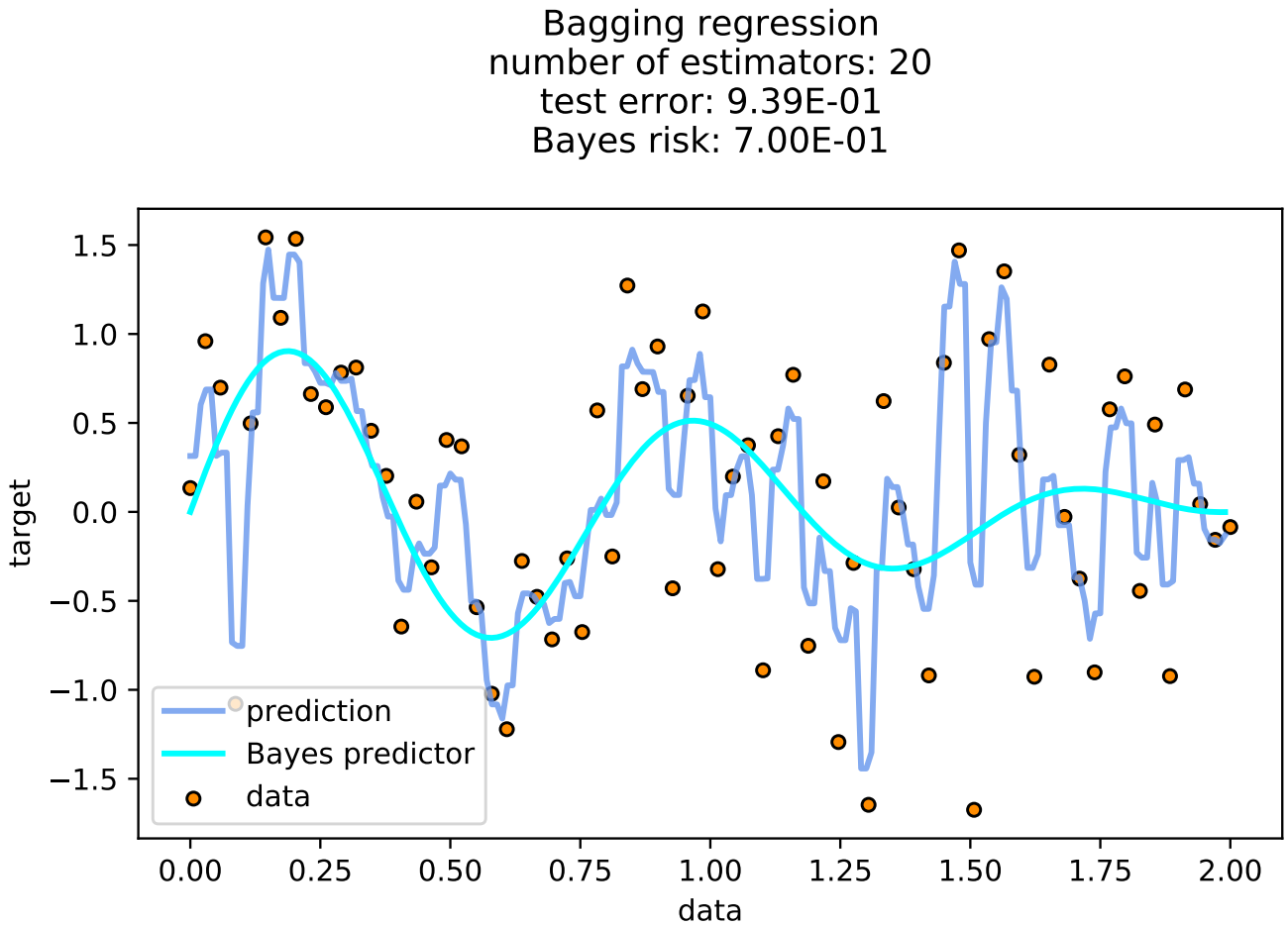
Simulation


Bagging regression
number of estimators: 1
test error: 1.22E+00
Bayes risk: 7.00E-01
target
data
prediction
Bayes predictor
data
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00

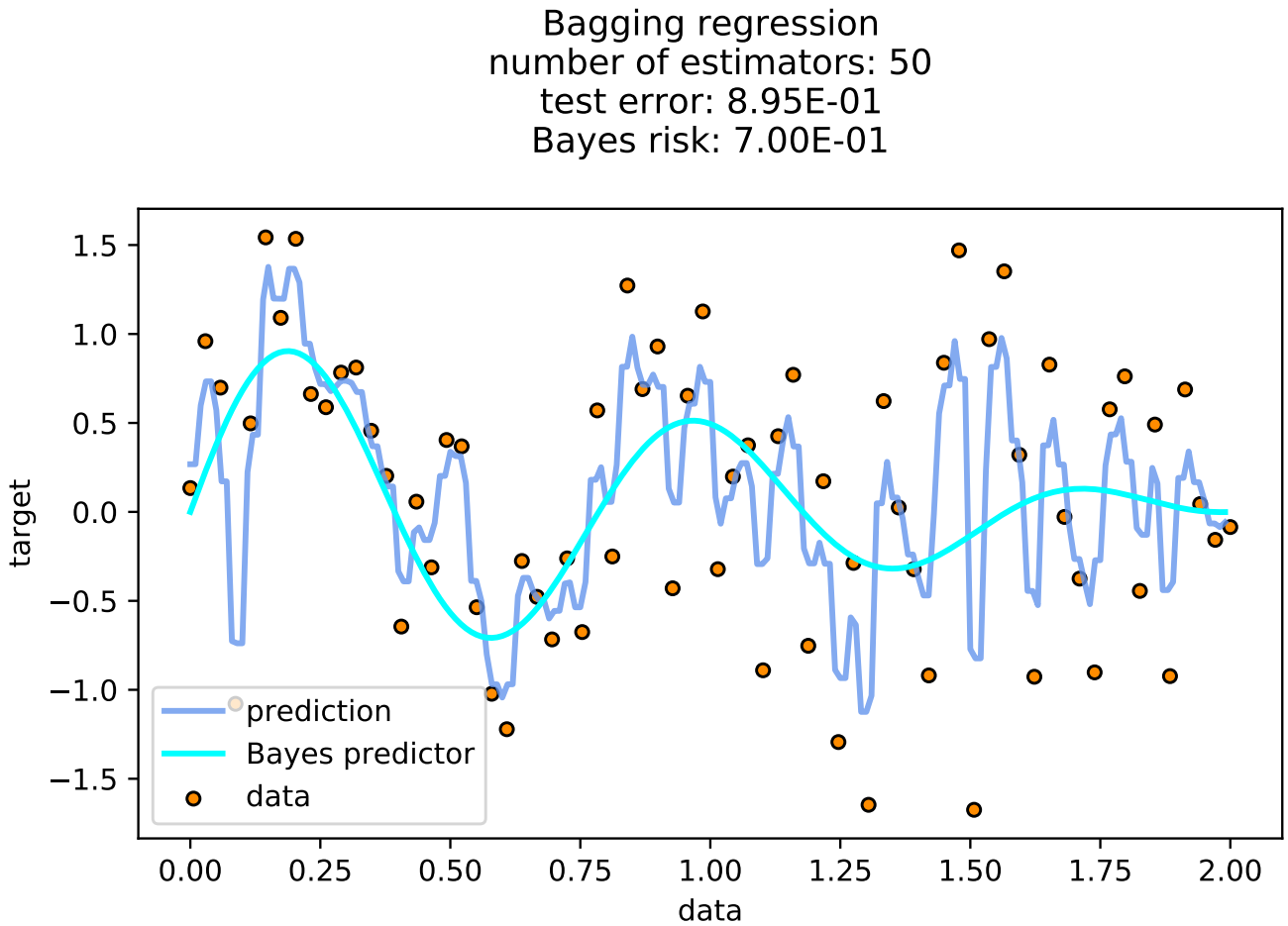
Simulation


Bagging regression
number of estimators: 10
test error: 9.09E-01
Bayes risk: 7.00E-01
target
data
prediction
Bayes predictor
data
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00

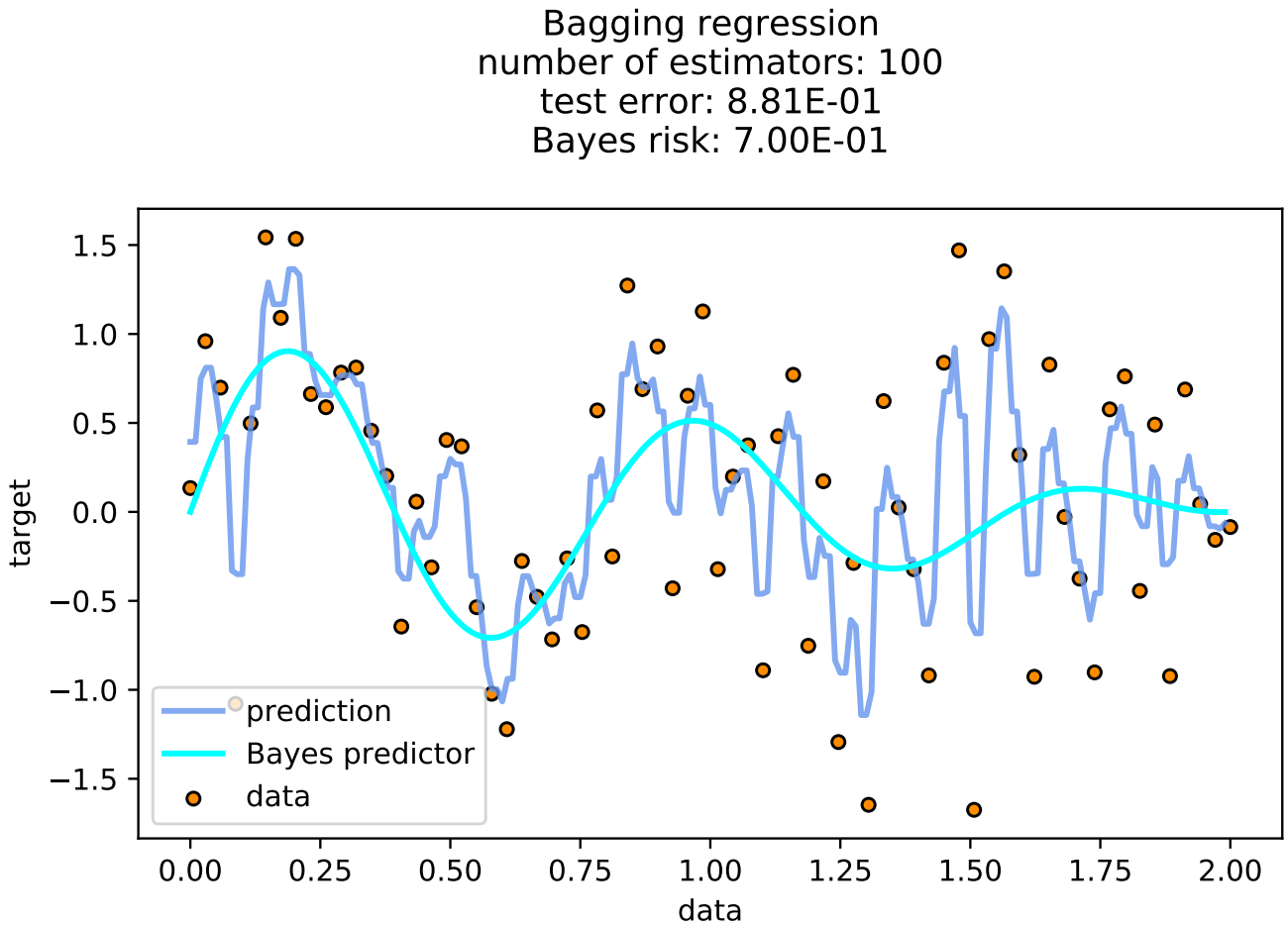
Simulation


Bagging regression
number of estimators: 20
test error: 9.39E-01
Bayes risk: 7.00E-01
target
data
prediction
Bayes predictor
data
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00

Simulation


Bagging regression
number of estimators: 50
test error: 8.95E-01
Bayes risk: 7.00E-01
target
data
prediction
Bayes predictor
data
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00

Simulation


Bagging regression
number of estimators: 100
test error: 8.81E-01
Bayes risk: 7.00E-01
target
data
prediction
Bayes predictor
data
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00

Individual estimators

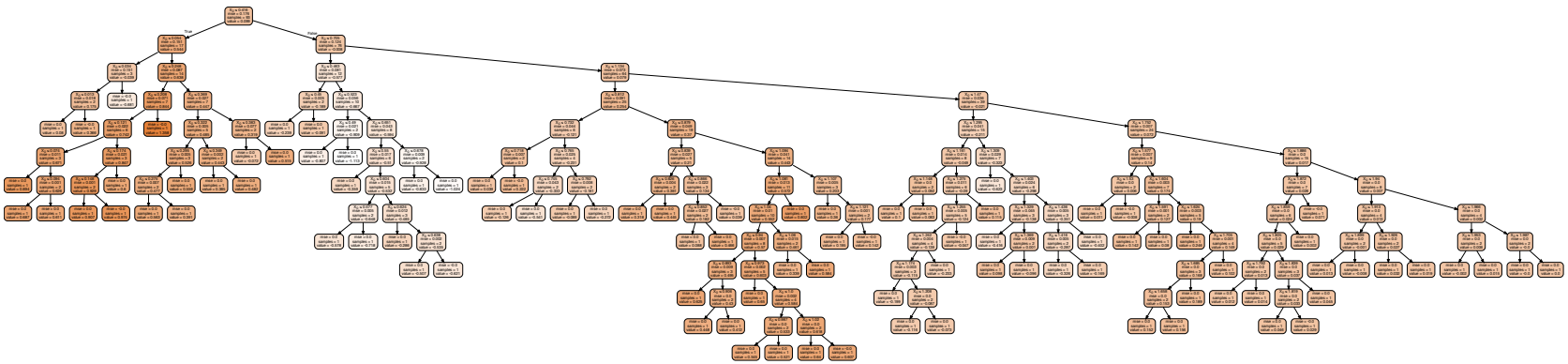


Figure – Estimator used in the averaging

Individual estimators

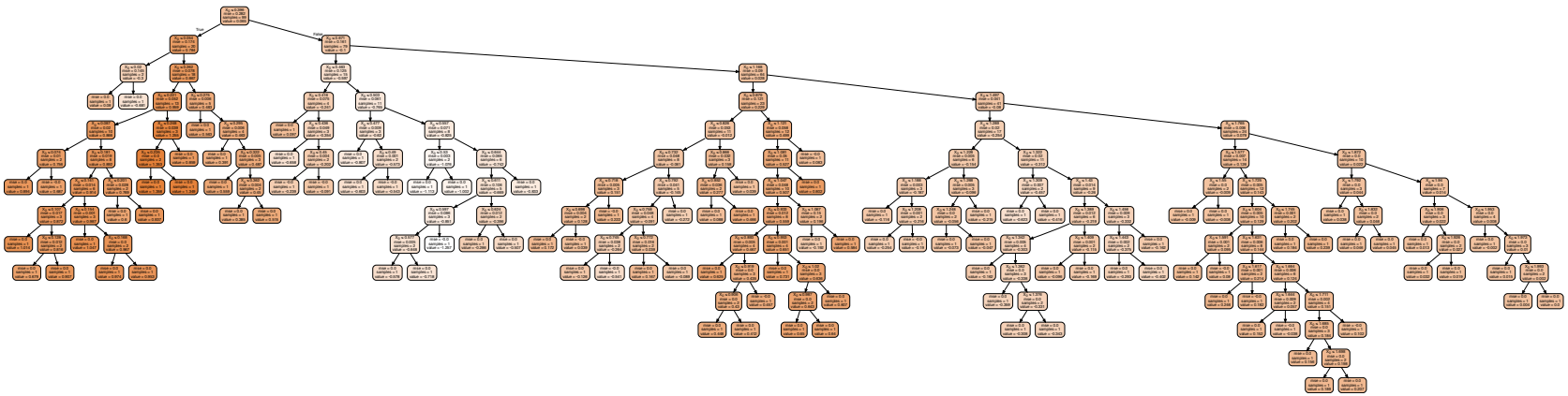


Figure – Other estimator used in the averaging

Possible issues

- number of trees to compute (B) in order to have a stable out-of-bag error.
- need for storing all the base estimators
- the final estimator is a *black-box* model.

Random forest

- Random forest [Breiman, 2001] is a bagging method with binary CART estimators, with additional randomness added to the choice of segmentation variables.
- the goal is to increase the **independence** between the base trees.
- although the theoretical properties of random forest are not yet fully understood, it is an important benchmark in supervised learning (but for instance not when the problem can be solved in a linear way).

Variance of an average : independent variables

Recall that if $(X_k)_{k\in\mathbb{N}}$ is a sequence of i.i.d. real variables that have a moment of order 2, and hence a variance σ^2, and an expected value of m. Then if $S_B = \frac{1}{B}\sum_{i=1}^{B} X_i$

$$\begin{aligned} Var(S_B) &= \sum_{i=1}^{B} Var(\frac{1}{B}X_i) \\ &= \frac{1}{B^2}\sum_{i=1}^{B} Var(X_i) \\ &= \frac{\sigma^2}{B} \end{aligned} \tag{34}$$

Variance of the average : correlated variables

However, if the X_i are identically distributed but correlated with correlation c, then we admit that

$$Vac(S_B) = c\sigma^2 + \frac{1-c}{B}\sigma^2 \tag{35}$$

Random forest diminishes c by adding some randomness in the choice of the segmentation variables.

Random forest

Let d be the number of features. Let $m \leq d$ be an integer.

Result: Random forest estimator

for $b \in [1, B]$ **do**

  Sample a bootstrap dataset z_b;

  Estimate $\hat{f}_{z_b}$ with **randomization** of the variables. The search of the optimal segmentation is done on m randomly sampled variables.;

end

$$\hat{f}_B = \frac{1}{B} \sum_{b=1}^{B} \hat{f}_{z_b} \tag{36}$$

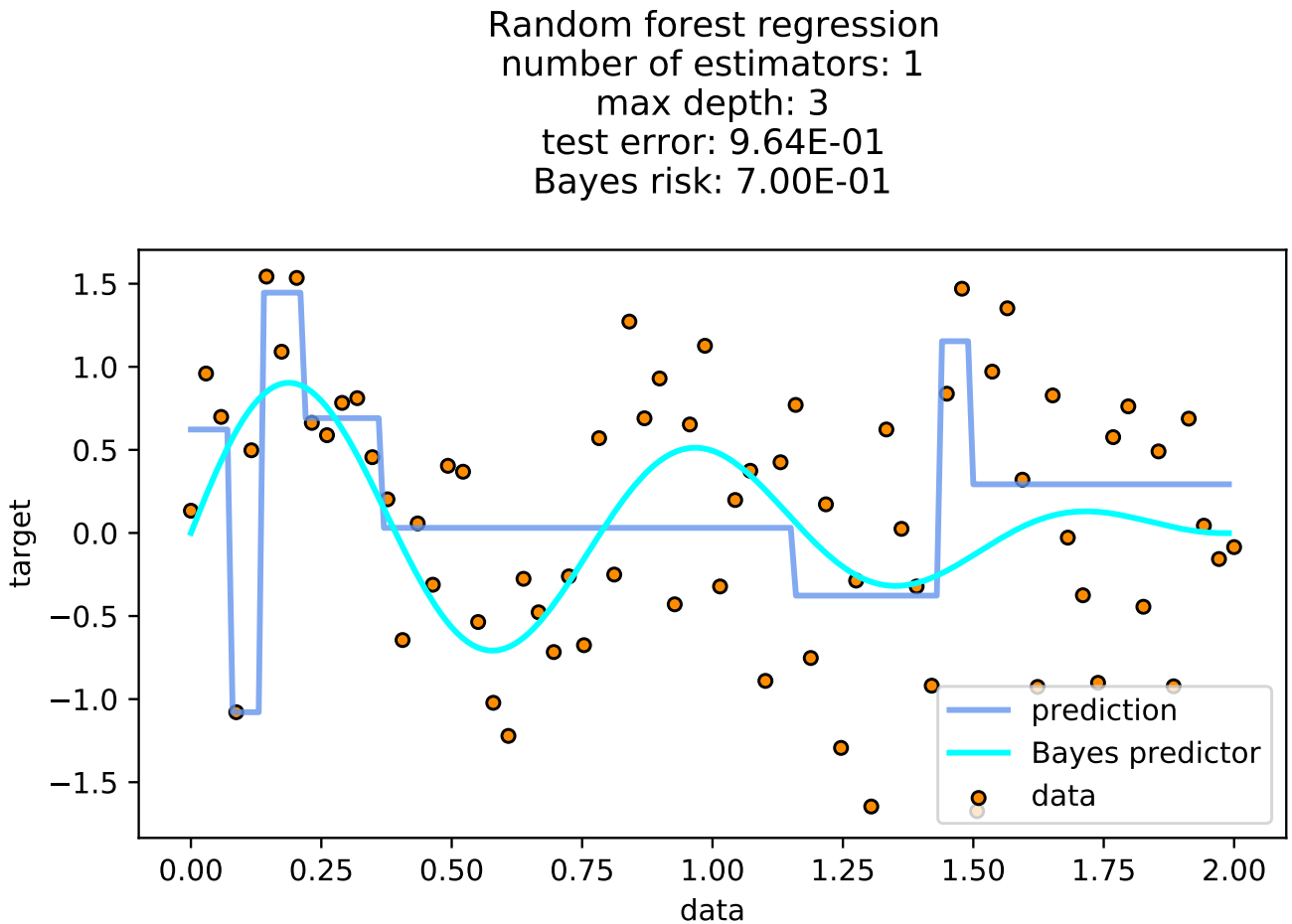
return $\hat{f}_B$

Algorithm 1: Random forest

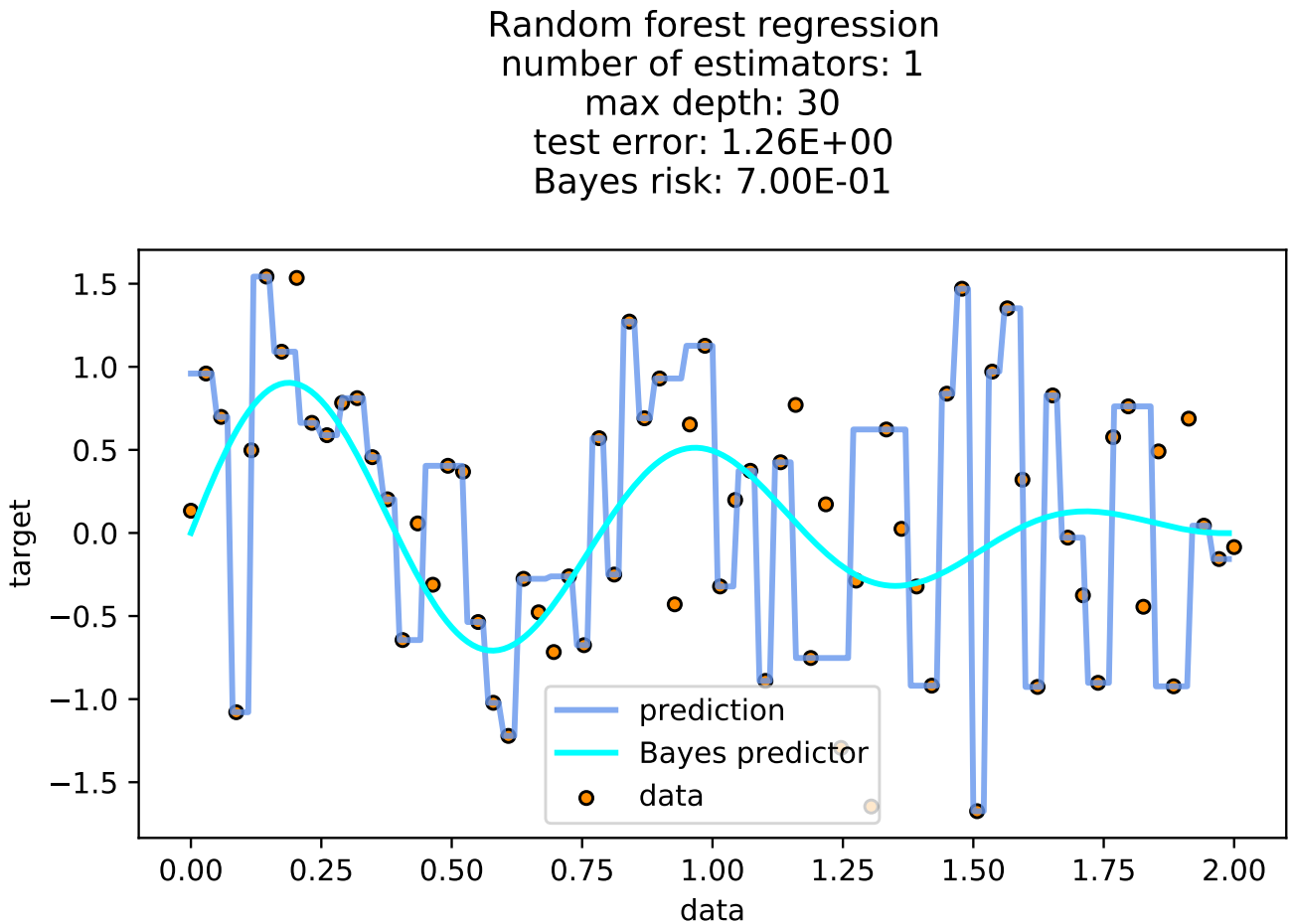
Remarks on random forest

- With random forest, it is possible to use a more brutal pruning (for instance using only trees of depth 2)
- To tune m, heuristics are used. Common ones include :
  - $m = \sqrt{d}$ for classification (d is the number of features)
  - $m = d/3$ for regression
  - cross validation.

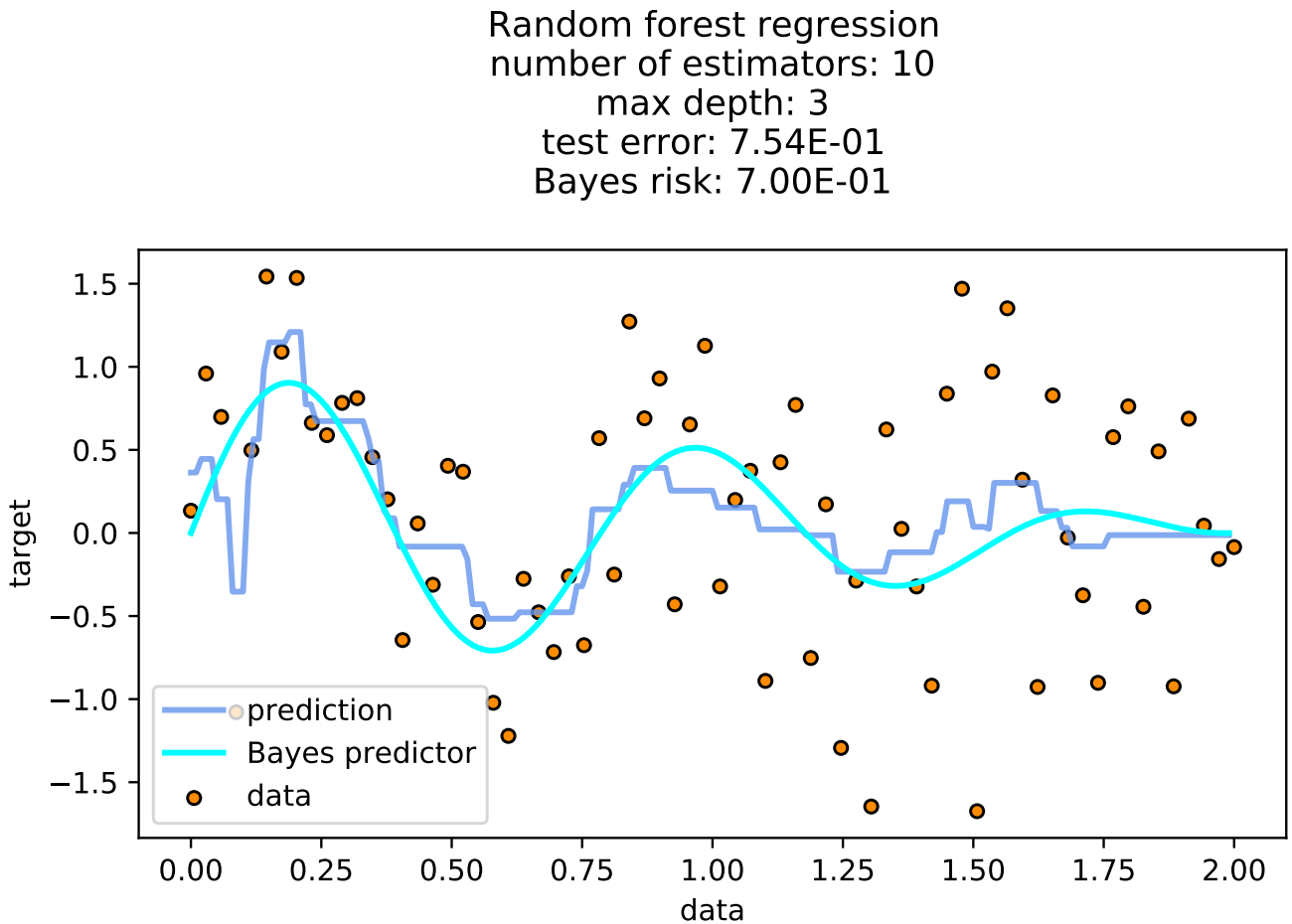
Example in 1D (not representative of actual random forests !)



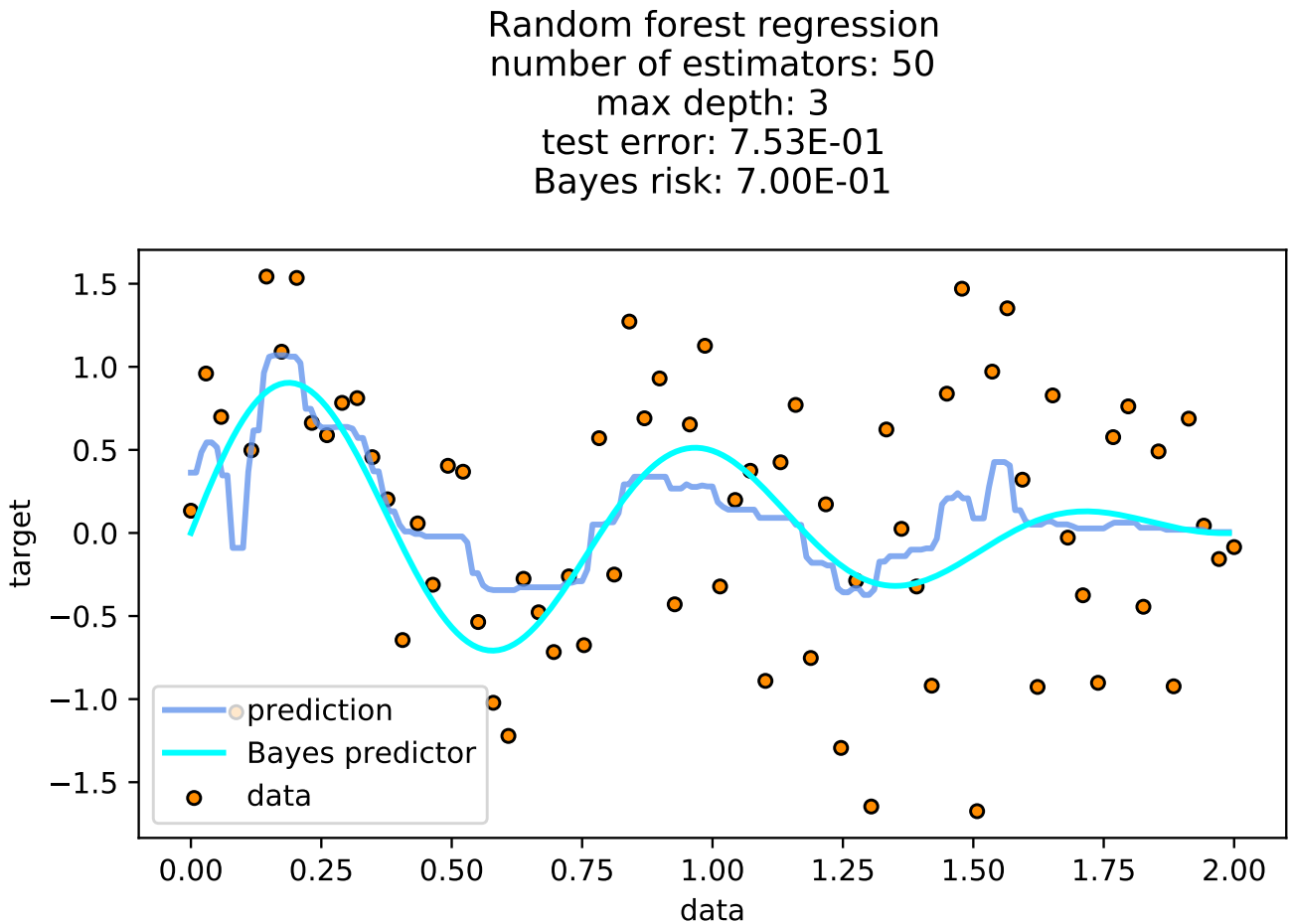
Example in 1D (not representative of actual random forests !)



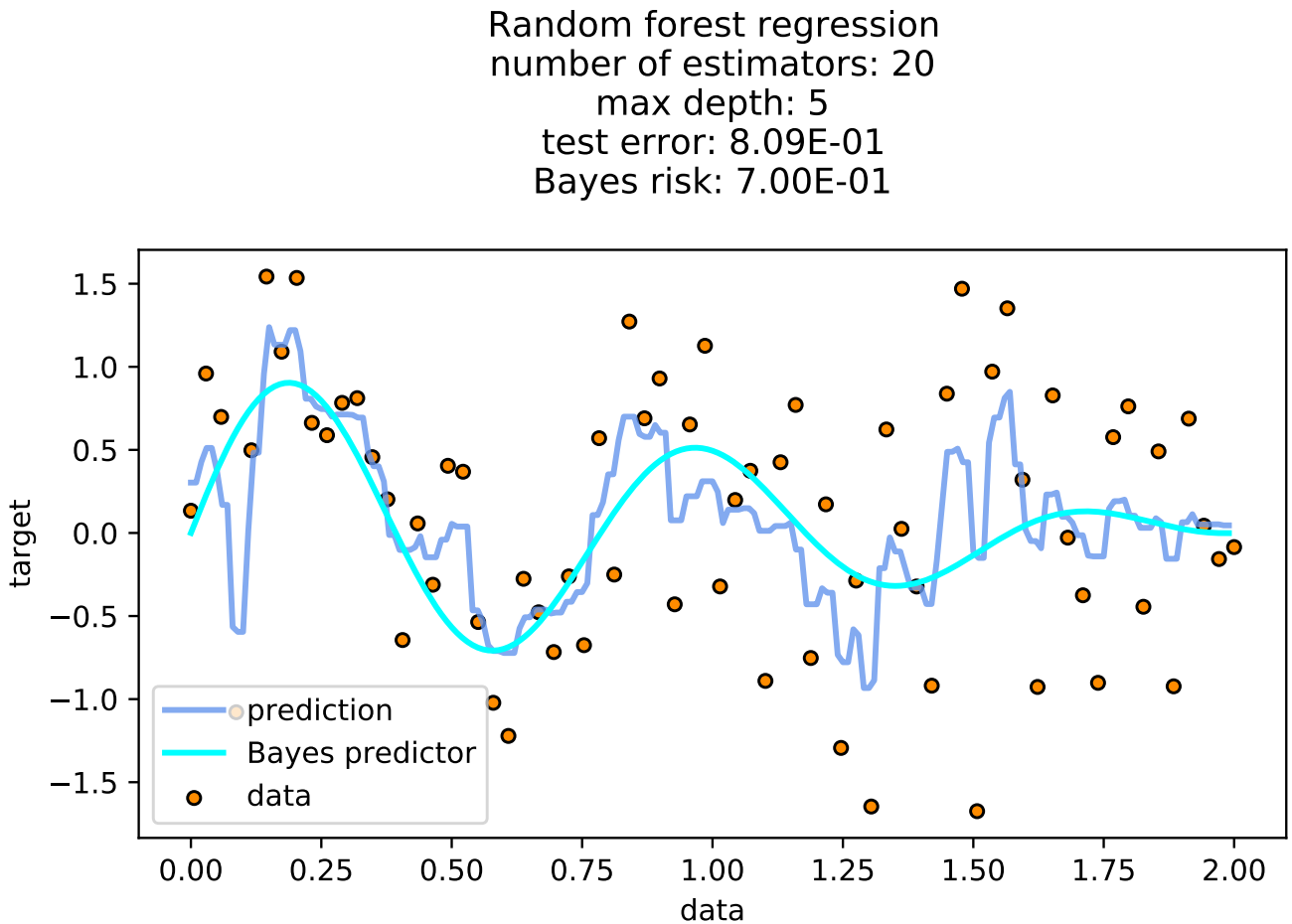
Example in 1D (not representative of actual random forests !)



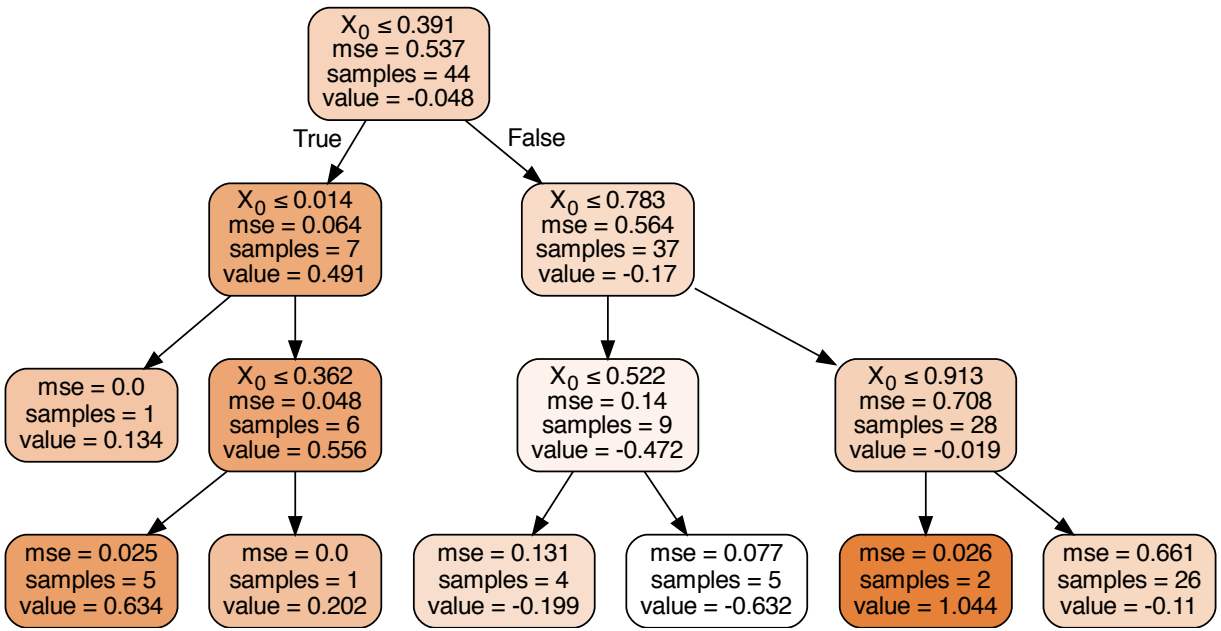
Example in 1D (not representative of actual random forests !)



Example in 1D (not representative of actual random forests !)



Estimators


$X_0 \leq 0.391$
mse = 0.537
samples = 44
value = -0.048
True
False
$X_0 \leq 0.014$
mse = 0.064
samples = 7
value = 0.491
$X_0 \leq 0.783$
mse = 0.564
samples = 37
value = -0.17
mse = 0.0
samples = 1
value = 0.134
$X_0 \leq 0.362$
mse = 0.048
samples = 6
value = 0.556
$X_0 \leq 0.522$
mse = 0.14
samples = 9
value = -0.472
$X_0 \leq 0.913$
mse = 0.708
samples = 28
value = -0.019
mse = 0.025
samples = 5
value = 0.634
mse = 0.0
samples = 1
value = 0.202
mse = 0.131
samples = 4
value = -0.199
mse = 0.077
samples = 5
value = -0.632
mse = 0.026
samples = 2
value = 1.044
mse = 0.661
samples = 26
value = -0.11

Importance of the variables

Although the obtained estimator is hard to interpret, we can still study the statistical importance of variables for the prediction.

Mean decrease accuracy

Consider a variable j. For a given tree b

- compute the out-of-bag error.
- shuffle the values of j in the out-of-bag sample.
- compute the new out-of-bag error
- store the decrease in prediction quality, D_j^b.

Average the D_j^b on b in order to measure the importance of the variable j.

Boosting

- While bagging is a random method, **boosting** is an adaptive method.
- Boosting builds on *weak classifiers*, and like bagging, aggregates them, for instance with a **weighted sum**.
- However, the weak classifiers are built **sequentially**. The classifier $b+1$ adapts the classifier b by focusing on improving the prediction of incorrectly classified training samples.
- Several variants exists (in the weighting, loss function, aggregating method, etc).

Adaboost

- Original boosting algorithm [Freund and Schapire, 1996]
- $\mathcal{Y} = \{-1, 1\}$ (but can also be adapted to a regression problem)
- Given the sample dataset $z = \{(x_1, y_1), \dots (x_n, y_n)\}$. We learn a **sequence of estimators** $(\delta_m)_{m \in [1..B]}$ (often CART).

Initialize the weights $w = \{w_i = \frac{1}{n}, i = 1..n\}$;
for $m = 1..B$ **do**

Optimize estimator δ_m on the weighted dataset. Compute the error rate

$$\hat{\epsilon_m} = \frac{\sum_{i=1}^{n} w_i 1(\delta_m(x_i) \neq y_i)}{\sum_{i=1}^{n} w_i} \tag{37}$$

Compute the logit of $\hat{\epsilon_m}$

$$c_m = \log\left(\frac{1 - \hat{\epsilon_m}}{\hat{\epsilon_m}}\right) \tag{38}$$

Update the weights

$$w_i \leftarrow w_i \exp\Big(c_m 1(\delta_m(x_i) \neq y_i)\Big) \tag{39}$$

end

$$\hat{\delta_B} = \text{sign}\Big(\sum_{b=1}^{B} \delta_{z_b}\Big) \tag{40}$$

return $\hat{f_B}$

Algorithm 2: Adaboost for binary classification (adaptive boos-

Remarks

We need to enforce that $c_m \geq 0$. It is verified if $\hat{\epsilon_m} \leq \frac{1}{2}$.

$$c_m = \log\left(\frac{1 - \hat{\epsilon_m}}{\hat{\epsilon_m}}\right) \tag{41}$$

- It is experimentally shown that if the weak classifiers are *stump trees* (2 leaves), then adaBoost performs better than one given tree with a number of leaves equal to the number of iterations of adaBoost (hence a comparable computation time).
- A number of leaves q between 4 and 8 for the weak classifiers is often recommended.
- adaBoost can be adapted to multiclass classification and regression [Schapire, 2003].

Gradient tree boosting

- differentiate with respect to the predictions of the tree (compute a gradient)
- approximate the gradient by a regression tree
- update the tree following this gradient.

Gradient tree boosting

Initialize $f_0 = \arg\min_z \sum_{i=1}^{n} l(y_i, z)$;
for $m = 1..B$ **do**
  Compute $r_{m_i} = -\frac{\partial l(y_i, f(x_i))}{\partial f(x_i)}(f = f_{m-1}), i \in [1, n]$
  Train a decision tree δ_m on $(x_i, r_{m_i})_{i \in [1,n]}$
  Compute γ_m minimizing

$$\gamma \to \sum_{i=1}^{n} l(y_i, f_{m-1}(x_i) + \gamma \delta_m(x_i)) \tag{42}$$

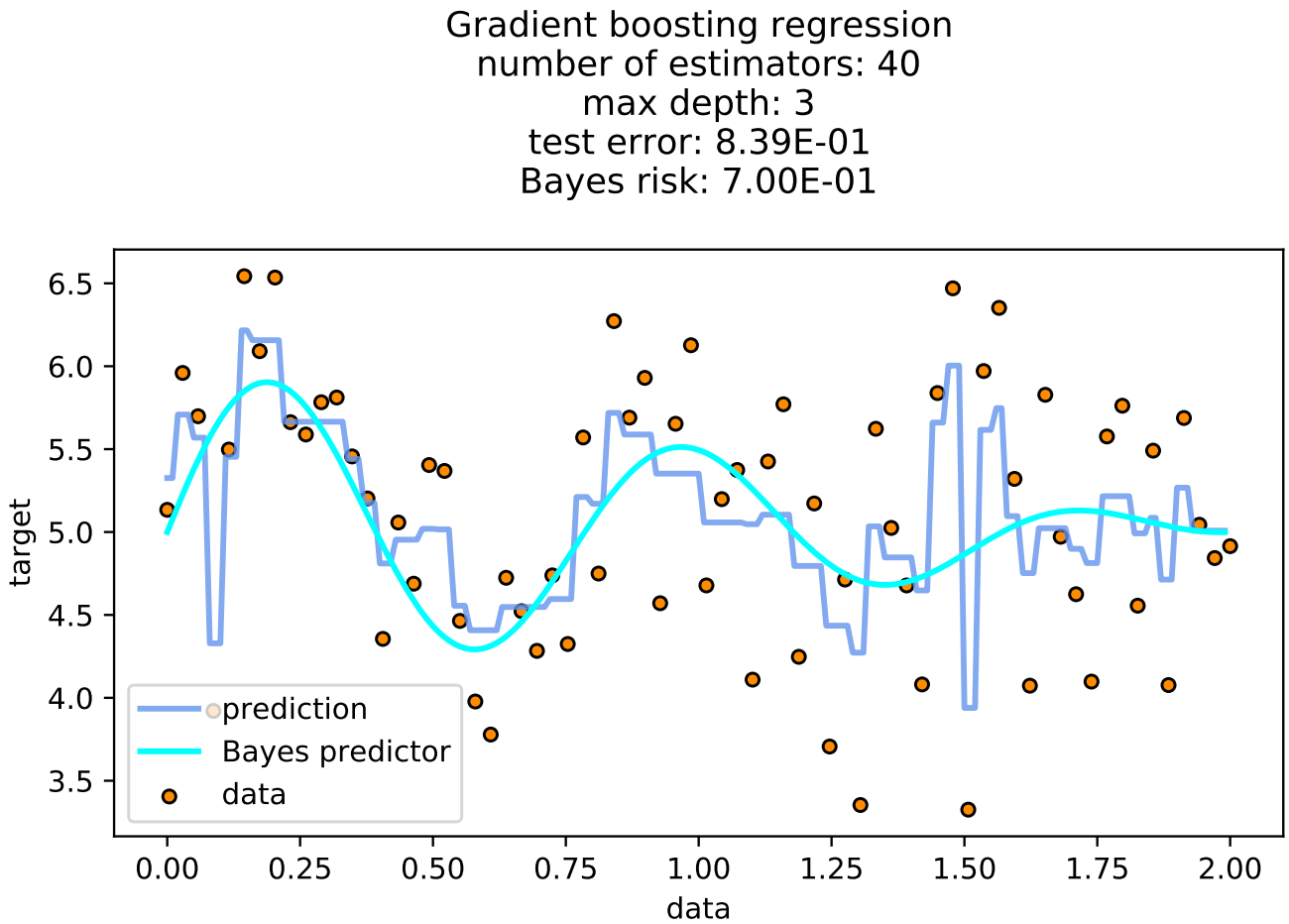
  Update the estimator

$$\hat{f_m} = \hat{f_{m-1}} + \gamma_m \delta_m \tag{43}$$

end
return $\hat{f_B}$

Algorithm 3: Gradient tree boosting

Simulation


Gradient boosting regression
number of estimators: 40
max depth: 3
test error: 8.39E-01
Bayes risk: 7.00E-01
target
data
prediction
Bayes predictor
data
6.5
6.0
5.5
5.0
4.5
4.0
3.5
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00

Gradient tree boosting

Many parameter can be tuned :

- maximum depth of the estimators
- shrinkage η ($\hat{f_m} = \hat{f_{m-1}} + \eta\gamma_m\delta_m$, with $0 \leq \eta \leq 1$). If η is low, e.g. ≤ 0.1, it can lead to a slower convergence but might prevent overfitting.
- number of trees learned B

`https://www.analyticsvidhya.com/blog/2016/02/complete-guide-parameter-tuning-gradient-boosting-gbm-pytho`

It is necessary to experiment, read documentations, cross validate, use heuristics, etc.

Extreme gradient boosting

- XGBoost : variant of gradient boosting, very efficient on several benchmarks [Chen and Guestrin, 2016]
- can exploit parallelization
- `https://xgboost.readthedocs.io/en/latest/parameter.html`
- `https://github.com/dmlc/xgboost`

Catboost

- See also : Catboost
  `https://catboost.ai/`

References I


Azencott, C.-A. (2022).
Introduction au Machine Learning - 2e éd.


Bottou, L. and Bousquet, O. (2009).
The tradeoffs of large scale learning.
Advances in Neural Information Processing Systems 20 - Proceedings of the 2007 Conference, (January 2007).


Breiman, L. (2001).
Random Forests.
Machine Learning, 45(1) :5–32.

References II


Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (2017).
Classification And Regression Trees.
Routledge.


Chen, T. and Guestrin, C. (2016).
XGBoost : A scalable tree boosting system.
Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 13-17-Augu :785–794.


Fernández-Delgado, M., Cernadas, E., Barro, S., and Amorim, D. (2014).
Do we need hundreds of classifiers to solve real world classification problems ?
Journal of Machine Learning Research, 15 :3133–3181.

References III


Freund, Y. and Schapire, R. E. (1996).
Experiments with a New Boosting Algorithm.
Proceedings of the 13th International Conference on Machine Learning, pages 148–156.


Schapire, R. E. (2003).
The Boosting Approach to Machine Learning : An Overview
BT - Nonlinear Estimation and Classification.
Nonlinear Estimation and Classification, 171(Chapter 9) :149–171.